

LECTURE 5 • PART I

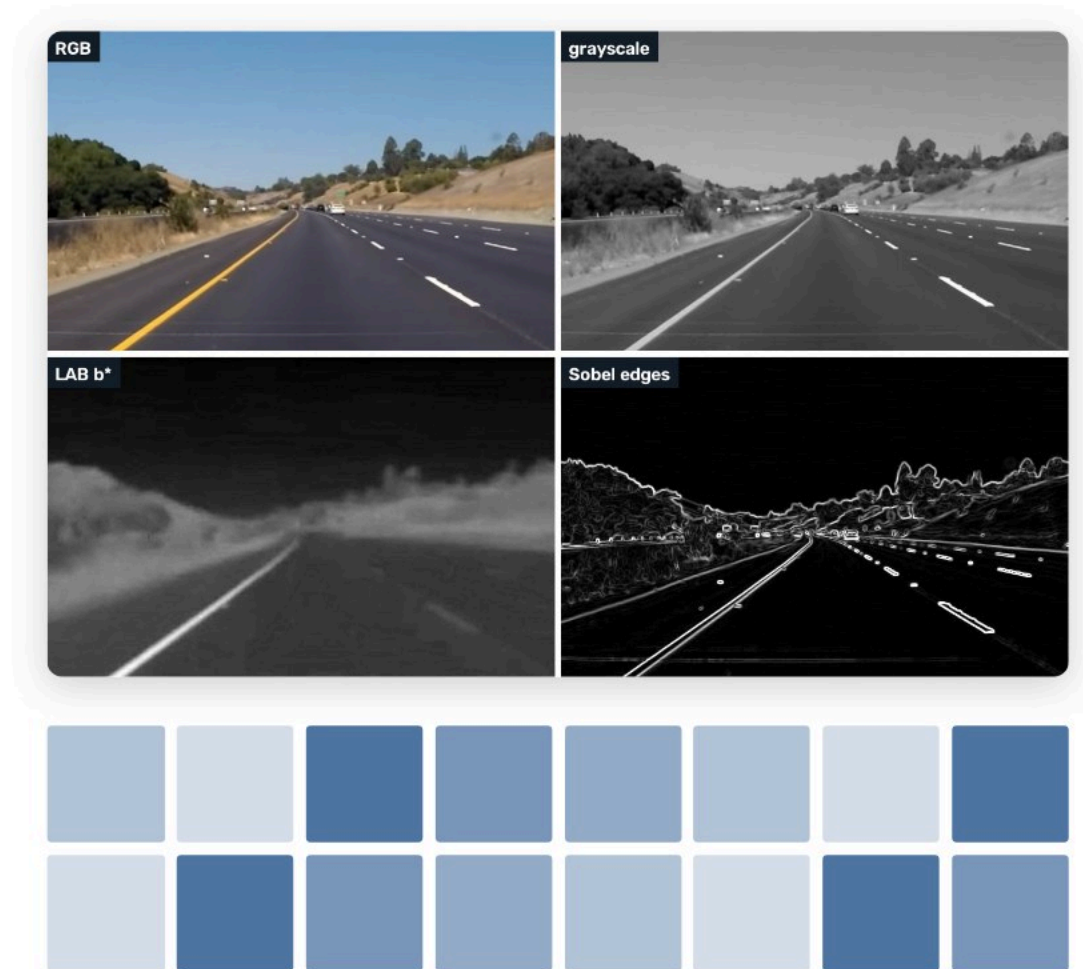
Computer Vision

Images, Colour Spaces, Filters & Features

With practical implementation in OpenCV • from pixels to deep learning

Dr. Muhammad Kazim

Department of Intelligent Systems
University of Lahore



Learning outcomes

By the end of this lecture you should be able to:

1. Explain how a computer stores an image: **pixels**, `uint8` values 0–255 and $H \times W \times C$ arrays (tensors).
2. Convert between **colour spaces** (RGB/BGR, HSV, LAB, YCbCr) and pick the right one for a task.
3. Explain **convolution** with a kernel and compute an averaging filter **by hand**.
4. Choose between **averaging**, **Gaussian** and **median** blur for a given type of noise.
5. Find **edges** (Sobel, Laplacian, Canny), **contours** and simple **shapes** with OpenCV.
6. Explain **features** (keypoints + descriptors), match them with **ORB**, and contrast hand-crafted with **learned** (deep) features.

Roadmap: from pixels to deep learning

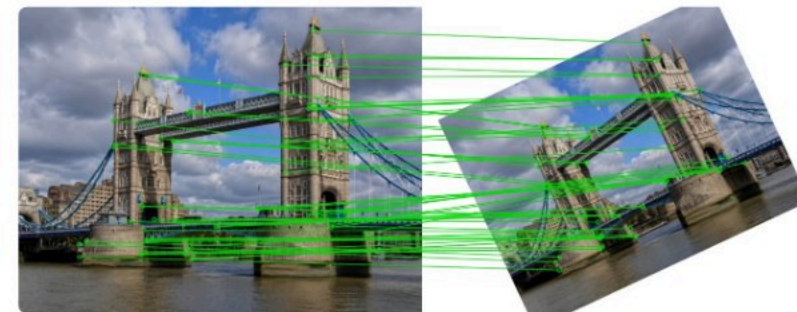


85	84	85	83	81	82	81	80	79	79	78	80	79	82
83	82	82	81	80	79	79	78	78	79	80	81	83	85
78	78	78	79	79	78	78	78	79	80	81	84	85	84
78	78	78	78	79	78	78	79	81	83	85	87	84	82
75	76	76	77	77	78	78	79	82	84	84	88	90	132
75	76	78	79	80	81	82	85	85	84	75	96	153	194
77	78	79	79	82	82	86	88	83	79	113	168	200	204
77	78	88	81	82	86	84	83	90	135	182	203	201	194
78	79	81	83	84	84	84	102	156	191	209	200	191	163

② every pixel is a number



④–⑤ filters reveal edges and shapes



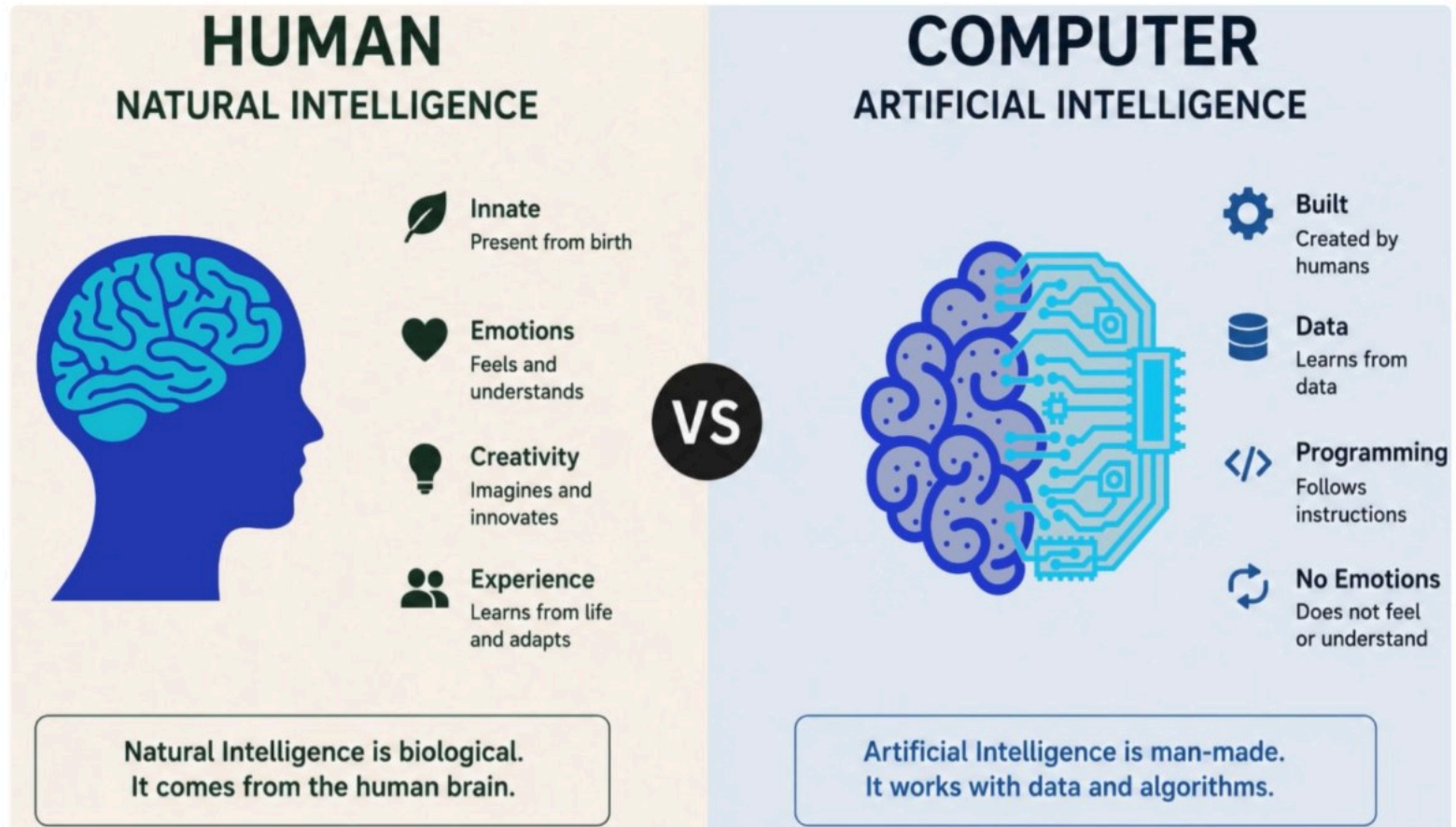
⑥ features find the same object again

01

Human vs computer vision

We see a mountain. What does the computer see?

Natural vs artificial intelligence



Natural intelligence

- Biological — present from birth
- Understands, imagines, feels
- Learns from a **few** experiences

Artificial intelligence

- Built by humans
- Follows **algorithms** (programs)
- Learns from **lots of data**

Computer vision is the part of AI that turns **pixels** into **understanding**: what is in the image, where it is, and what is happening.

Human vision vs computer vision



You see a green ridge, mist and mountains — instantly and without effort — your brain has learned to see over a lifetime.

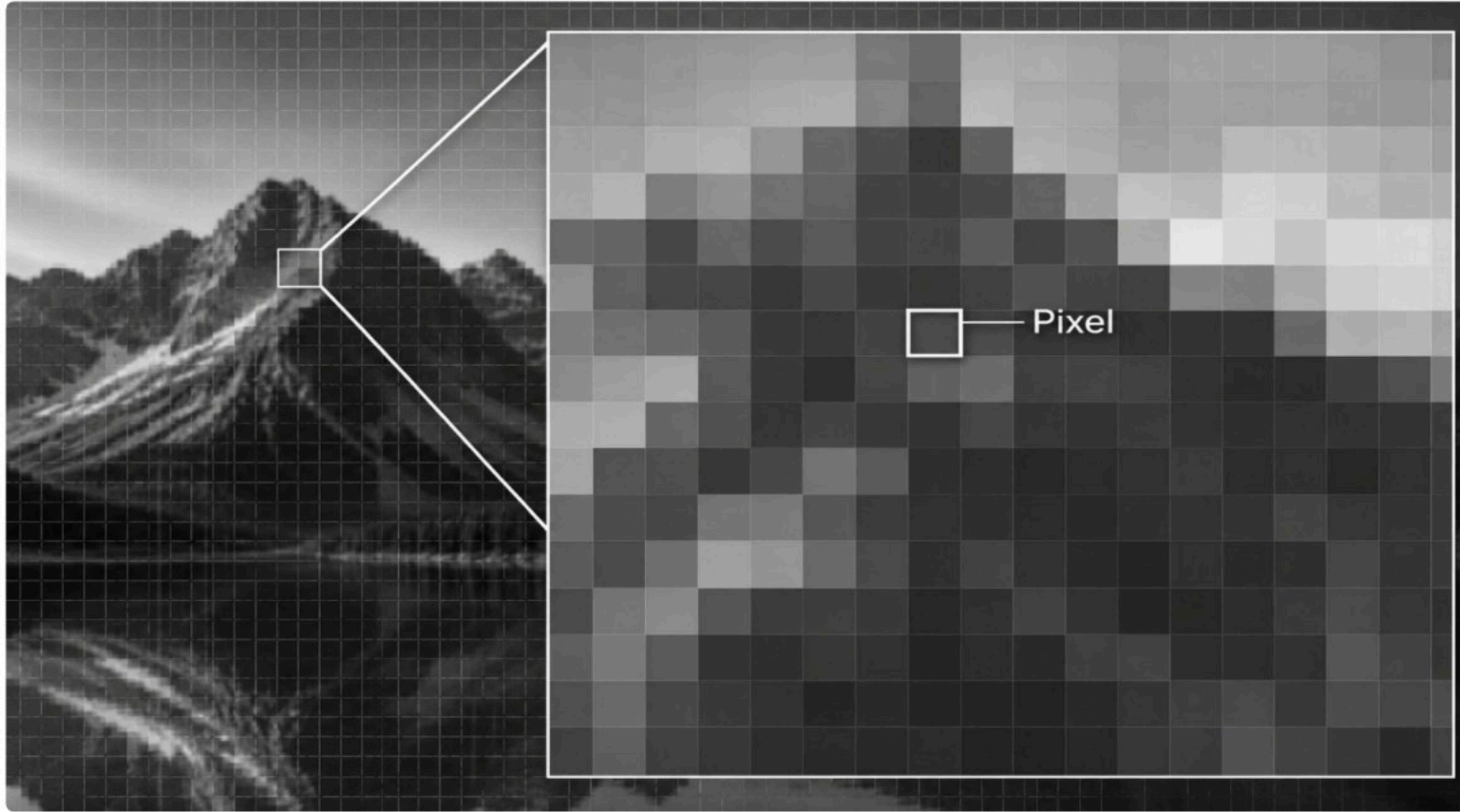
The computer receives only a **grid of numbers**. Every task — classify, detect, segment, track — starts from those numbers.

02

Images are numbers

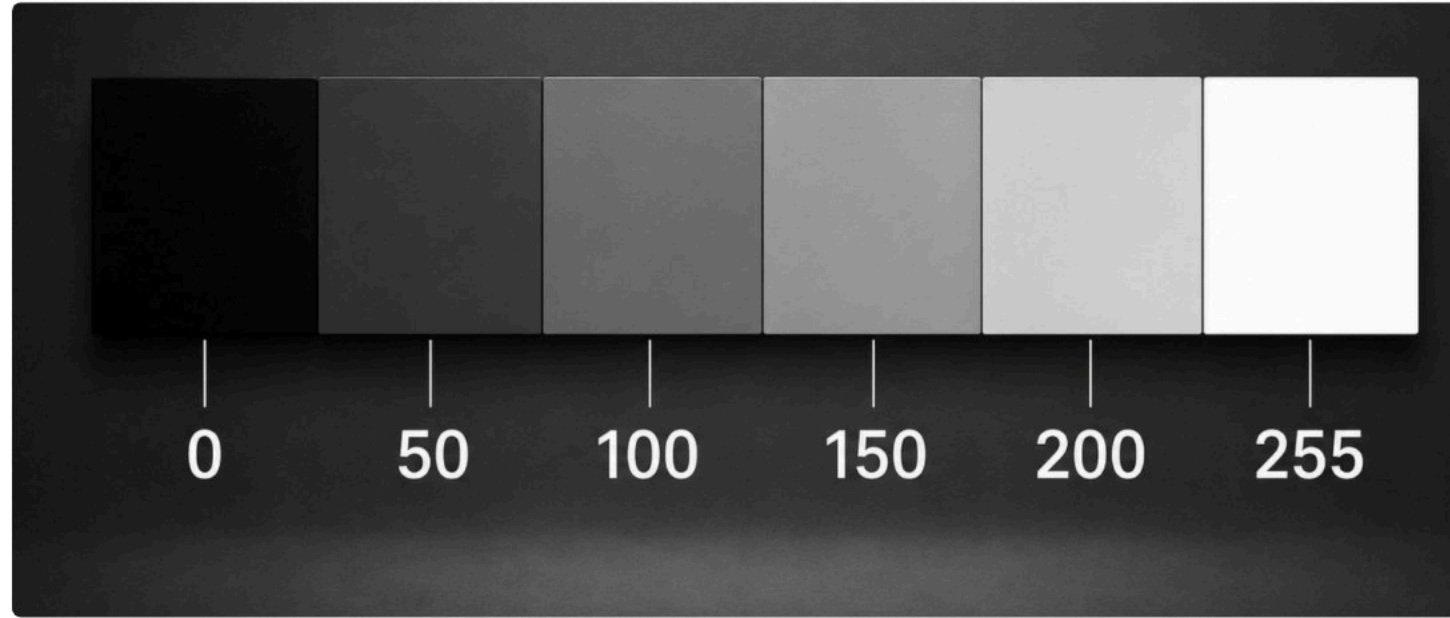
Pixels, intensity, colour channels and tensors

Zoom in: an image is made of pixels



- A digital image is a **grid of tiny squares** called **pixels** (picture elements).
- Each pixel stores **one value** (grayscale) or **three values** (colour).
- **Resolution** = width $\times$ height.
Full HD: $1920 \times 1080 = 2,073,600$ pixels.
- Zoom in far enough and the smooth mountain becomes visible squares.

Grayscale intensity: 0 to 255



Why 0–255?

Each pixel uses **8 bits** = 2^8 = **256** levels. **0 = black**, **255 = white**, everything else is a shade of gray. In NumPy the type is `uint8`.

Overflow trap. `uint8` wraps around: `np.uint8(250) + 10` → 4. OpenCV saturates instead: `cv2.add` gives 255. Convert to `float32` before doing arithmetic.

A real image patch, as numbers

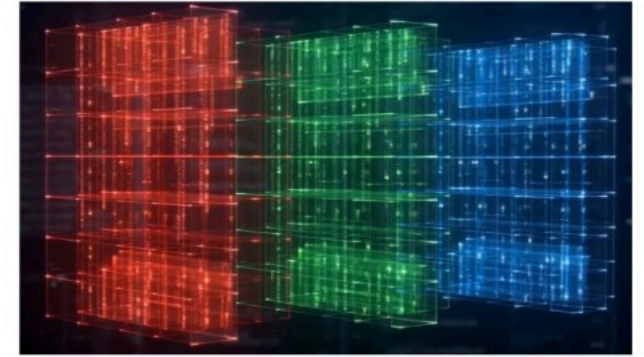
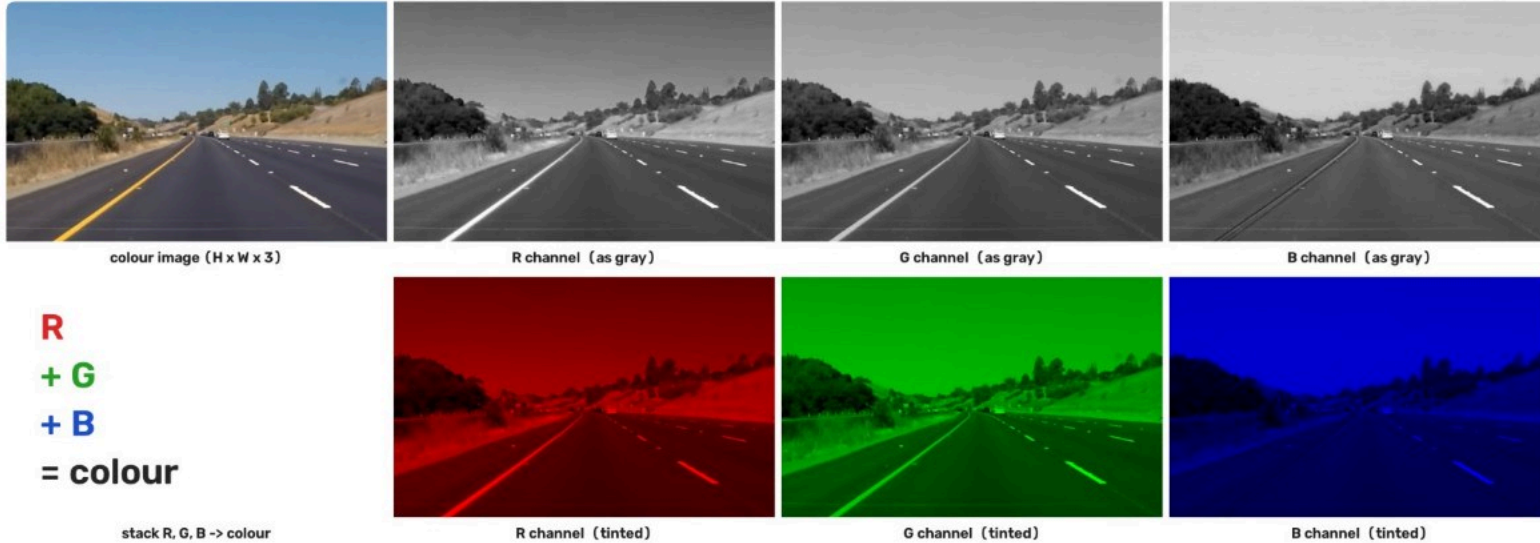


85	84	85	83	83	82	81	80	79	79	78	80	79	82
83	82	82	81	80	79	79	78	78	79	80	81	83	85
78	78	78	79	79	78	78	78	79	80	81	84	86	86
78	78	78	78	79	78	78	79	81	83	85	87	84	85
75	76	76	77	77	78	78	79	82	86	84	80	90	132
75	76	78	79	80	81	82	85	85	84	75	96	153	194
77	78	79	79	82	82	86	88	81	79	113	168	200	204
77	78	80	81	82	86	84	83	90	135	182	203	201	194
78	79	81	83	84	84	84	102	156	191	205	200	191	163

```
import cv2
img = cv2.imread("road.jpg", cv2.IMREAD_GRAYSCALE)
print(img.shape, img.dtype)    # (847, 1344) uint8
print(img[606, 452])          # row (y) first, then column (x)
patch = img[606:633:3, 452:494:3]
```

The red box covers road and lane paint. Road pixels are about **80**; the paint jumps to about **200**. A sudden jump like this is an **edge** — that is exactly what the filters in section 4 will find.

Colour images: three channels



- A colour pixel = (R, G, B), three 8-bit numbers.
- Each channel is a grayscale image on its own.
- The **yellow** line is bright in R and G but **dark in B** — yellow = red + green.
- OpenCV stores channels in B, G, R order!

$256 \times 256 \times 256 = 16,777,216$ colours



16777216
COLORS

3 channels $\times$ 8 bits = **24-bit “true colour”**,
enough for more shades than the eye can tell
apart.

How big is one frame?

$1920 \times 1080 \times 3$ bytes = 6,220,800
bytes $\approx$ 6.2 MB

Video at 30 fps $\rightarrow \approx$ 187 MB every
second, uncompressed.

That is why images and video are
compressed (JPEG, H.264) — and why
compression uses the **YCbCr** colour
space you will meet shortly.

Images as tensors

COMPUTER VISION

Understanding Images as Tensors

Images might look like continuous scenes, but computers see them as numbers arranged in grids. This mathematical representation is the foundation of modern computer vision.



1. Grayscale Image: A 2D Grid

A grayscale image consists of a 2D grid of pixels. Each pixel holds a single intensity value between 0 (black) and 255 (white).



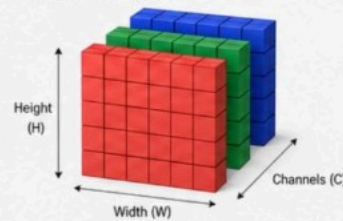
2. Color Image: Multiple Channels

A color image stores more information. Each pixel contains three values representing Red, Green, and Blue (RGB). These three channels combine to produce the final color.



3. From Grids to Tensors

Think of a color image as stacking multiple 2D grids together. The height and width define the spatial structure, and the channels add depth.



“In programming terms, we call 3D stack of grids a **Tensor**.”

A tensor is a multi-dimensional array of numbers. For a color image, the tensor shape is **(H, W, C)**, where C = 3 for RGB.

Why It Matters?

Representing images as tensors allows algorithms and neural networks to process visual data efficiently. Almost every computer vision model works directly on these tensor representations.



Deep Learning Models



Object Detection



Image Segmentation



Image Classification

A **tensor** is an n-dimensional array. Libraries disagree on the order of the axes:

Library	Shape	Values
OpenCV / NumPy	(H, W, C), BGR	<code>uint8</code> 0–255
Matplotlib / PIL	(H, W, C), RGB	<code>uint8</code>
PyTorch	(N, C, H, W)	<code>float32</code> 0–1

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
t = torch.from_numpy(rgb)           # (H, W, 3)
t = t.permute(2, 0, 1) / 255.      # (3, H, W)
batch = t.unsqueeze(0)             # (1, 3, H, W)
```

Quiz: images as numbers

QUESTION 1

`img.shape` returns `(480, 640, 3)`. How **wide** is the image?

640 px. NumPy shape is (rows = height, columns = width, channels).

QUESTION 2

How many bytes does one uncompressed **4K** RGB frame (3840×2160) need?

24,883,200 bytes $\approx$ 24.9 MB = $3840 \times 2160 \times 3$.

QUESTION 3

A pixel reads `(0, 255, 255)` in OpenCV. What colour is it?

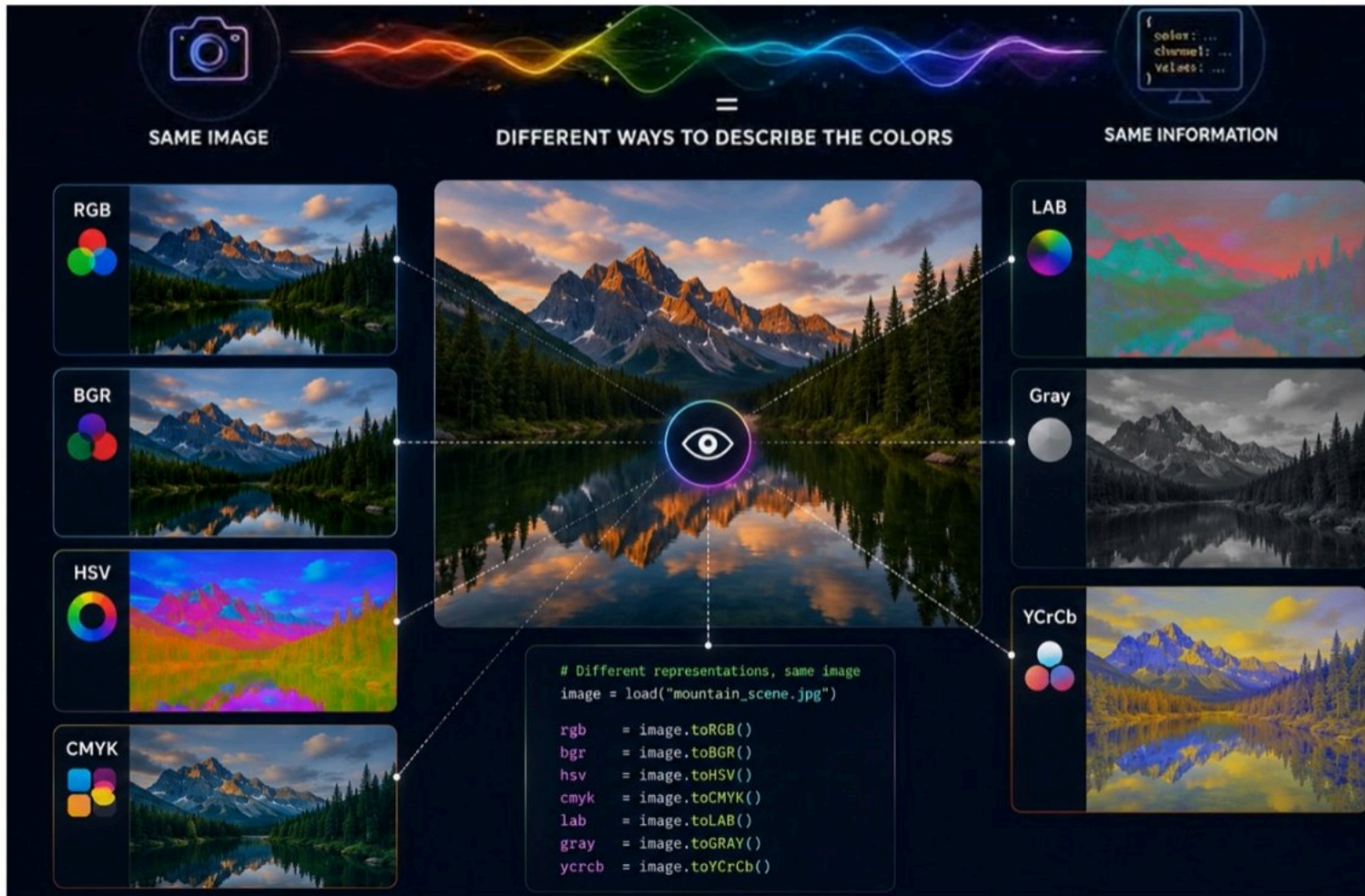
Yellow. OpenCV order is B, G, R $\rightarrow$ R = 255, G = 255, B = 0.

03

Colour spaces

One image, many ways to describe its colours

Same image, different descriptions



A **colour space** is a coordinate system for colour. Converting changes the **numbers**, not the **information**.

We switch spaces to **separate** what we care about:

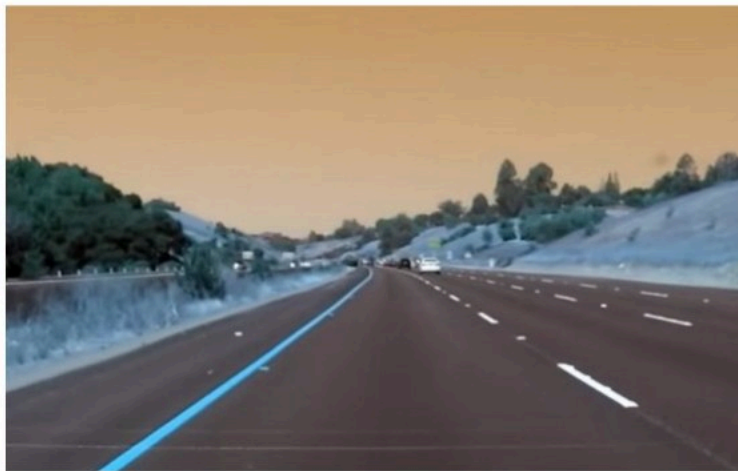
- **HSV / HLS** — colour (hue) apart from brightness
- **LAB** — distances match human perception
- **YCbCr** — brightness apart from colour, for compression
- **Gray** — one channel, when colour does not matter

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
```

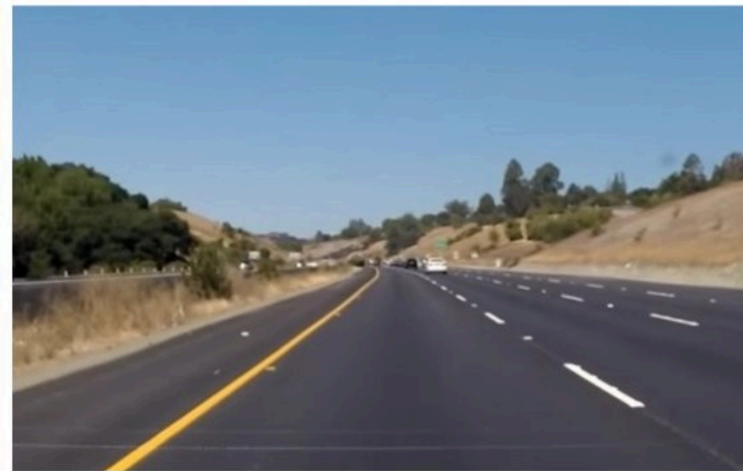
OpenCV's first gotcha: BGR, not RGB



`cv2.imshow(img)` -> correct



`plt.imshow(img)` -> R and B swapped!



`plt.imshow(rgb)` -> correct

```
img = cv2.imread("road.jpg") # B, G, R
plt.imshow(img)               # wrong colours

rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb)               # correct
```

OpenCV uses BGR for historical reasons: it was the common byte order of camera drivers and Windows bitmaps when the library was written. Everything else (Matplotlib, PIL, PyTorch models) expects **RGB**. Convert once, at the boundary.

Every space, channel by channel



HSV: H hue (0-179)



HSV: S saturation



HSV: V value



LAB: L* lightness



LAB: a* green-red



LAB: b* blue-yellow



YCrCb: Y luma



YCrCb: Cr red difference



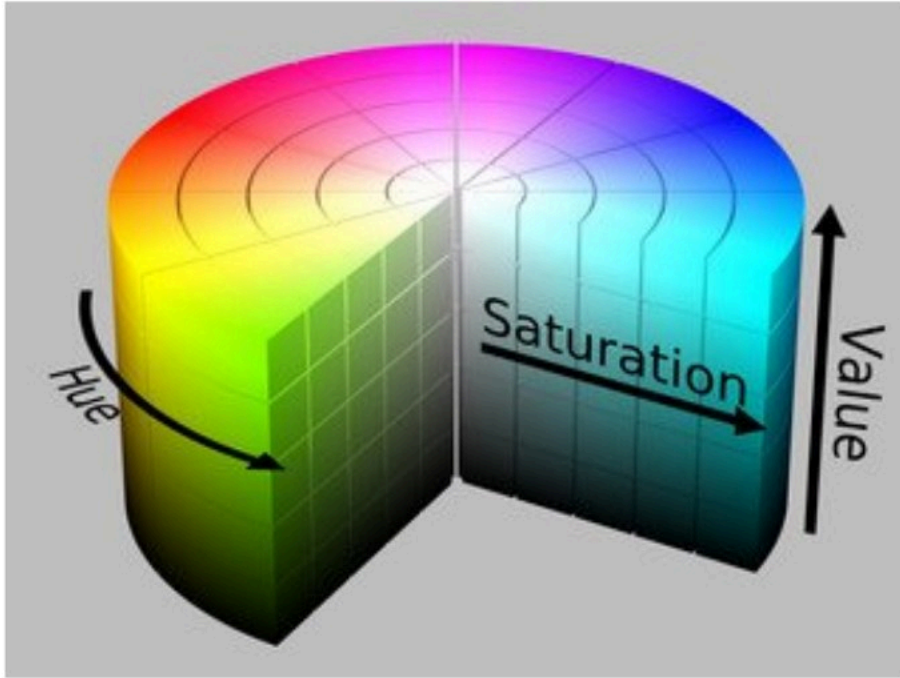
YCrCb: Cb blue difference

The same road in three colour spaces. Things to notice:

- **V, L*** and **Y** all look like a grayscale photo — they hold the brightness.
- **H** (hue) is flat across the whole road: hue ignores how bright things are.
- **S** (saturation) makes the colourful yellow line stand out against the gray road.
- **b*** and **Cr/Cb** separate yellow and blue: sky dark, yellow line bright.

Channels with a small range (H, a*, b*, Cr, Cb) are stretched to 0-255 for display.

HSV: hue, saturation, value



- **H — hue:** the colour itself, an **angle** around the cylinder (red → yellow → green → blue → red).
- **S — saturation:** how **pure** the colour is (0 = gray, max = vivid).
- **V — value:** how **bright** it is (0 = black).

OpenCV scaling: H is 0–179 (degrees ÷ 2, so it fits in a `uint8`); S and V are 0–255.

Why it helps: a yellow line in **shadow** changes V a lot but H very little, so thresholding H is much more robust than thresholding RGB.

HSV in practice: select the lane paint



1. original



2. inRange: yellow + white

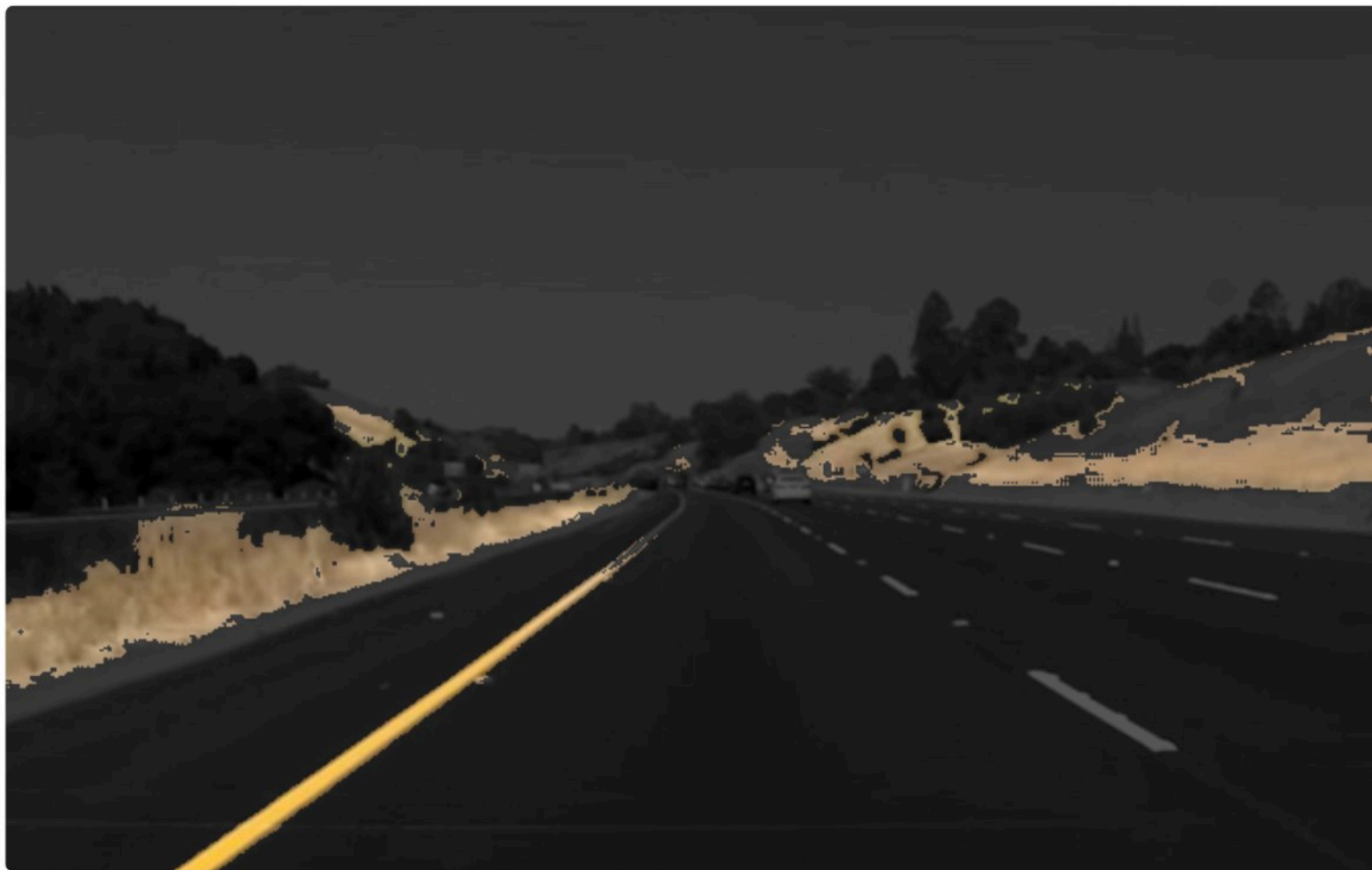


3. bitwise_and(img, img, mask)

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
# bounds are (H, S, V)
yellow = cv2.inRange(hsv, (15, 80, 120), (35, 255, 255))
white = cv2.inRange(hsv, (0, 0, 200), (179, 40, 255))
mask = cv2.bitwise_or(yellow, white)
result = cv2.bitwise_and(img, img, mask=mask)
```

Look closely: the dry grass is yellow too, so colour alone is not enough. In Part II we add a **region-of-interest** mask to keep only the road.

HSV threshold explorer



H min 15
H max 35
S min 80
S max 255
V min 120

yellow paint

white paint

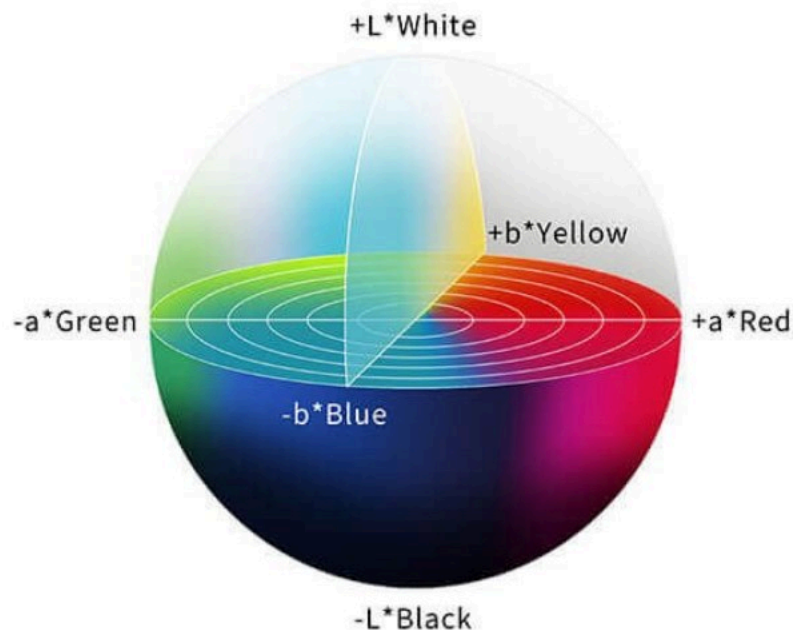
blue sky

green trees

☐ show the mask only

Selected: 6.7% of the pixels. H uses OpenCV's 0–179 scale.

LAB: a colour space built around human perception



- **L*** — lightness, black (0) $\rightarrow$ white (100)
- **a*** — green (-) $\leftrightarrow$ red (+)
- **b*** — blue (-) $\leftrightarrow$ yellow (+)

Perceptually uniform: the same distance in LAB looks like the same colour difference to a person (ΔE). Used for colour matching, printing quality checks and robust segmentation.

OpenCV 8-bit scaling: $L^* \times 255/100$, and a^* , b^* are shifted by +128 (so 128 = neutral gray).

Why it matters: finding a yellow line



original



grayscale: yellow ~ road



HLS S channel



LAB b* channel: yellow pops

Grayscale

Yellow paint ends up a similar gray to the road: **low contrast**, easy to miss in shadow.

HLS S channel

Saturated paint is bright, gray asphalt is dark — but the grass is bright too.

LAB b* channel

Yellow is large positive b*: the line **pops out** and the sky turns dark.

Application: lane finding with LAB

Radius: 365.0 m
Center departure: 0.27 m [Left]
Method: Fit margin

Lane finding using LAB color space

An advanced lane finder thresholds the **LAB** **b*** channel for yellow paint and the **L*** channel for white paint, fits curves to the lane pixels, and reports:

- radius of curvature (here 365 m)
- how far the car is from the lane centre

In [Part II](#) you build the simpler version step by step: colour selection → region mask → Canny → Hough lines.

YCbCr: brightness plus two colour differences

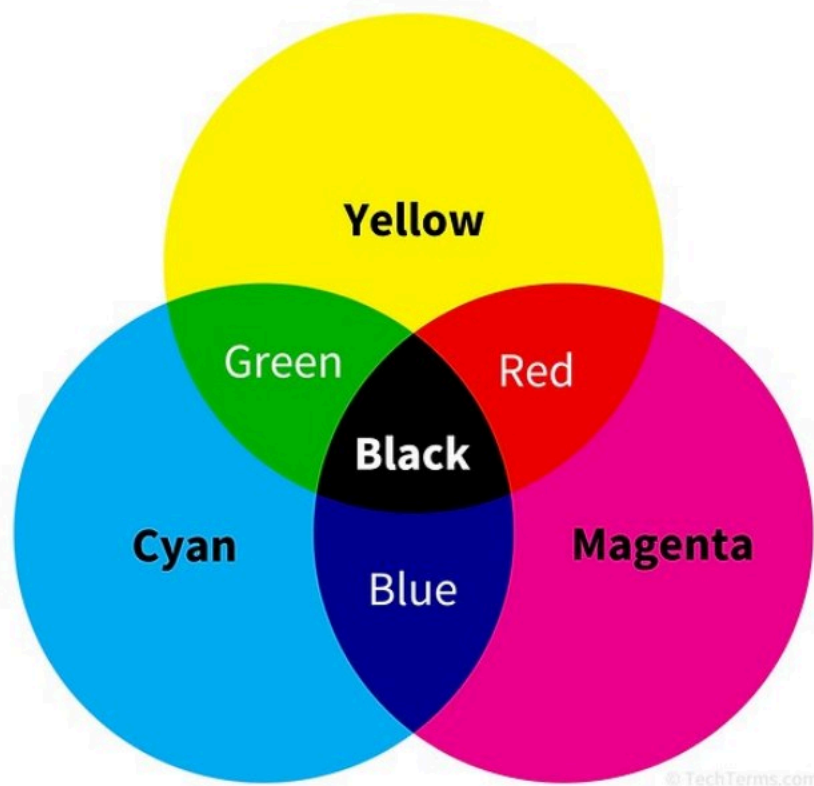


- $Y \text{ (luma)} = 0.299 R + 0.587 G + 0.114 B$ — the eye is most sensitive to green.
- **Cb** = blue – Y, **Cr** = red – Y (colour differences). OpenCV order: Y, Cr, Cb.

Where it is used

- **JPEG and video:** the eye sees less colour detail than brightness detail, so Cb and Cr are stored at lower resolution (4:2:0).
- **Skin detection:** skin tones form a tight cluster in Cb–Cr.

CMYK: colour for printers



- **Subtractive:** inks absorb light. Cyan + magenta + yellow → (almost) black.
- **K** = key (black) ink: a real black, cheaper and sharper text.
- Screens **emit** light → RGB is additive. Paper **reflects** light → CMYK.

CMYK is a **printing** space. OpenCV has no CMYK conversion and you will rarely use it in a vision pipeline — but you should recognise it.

Colour spaces: when to use which

Space	Channels	Good for	OpenCV
RGB / BGR	red, green, blue	display, storage, deep-learning input	<code>cv2.COLOR_BGR2RGB</code>
Gray	intensity	edges, features, speed (1 channel instead of 3)	<code>cv2.COLOR_BGR2GRAY</code>
HSV / HLS	hue, saturation, value / lightness	colour thresholding, robust to shadows	<code>cv2.COLOR_BGR2HSV</code> · <code>HLS</code>
LAB	lightness, green–red, blue–yellow	perceptual colour difference, yellow/blue separation	<code>cv2.COLOR_BGR2LAB</code>
YCbCr	luma, blue diff., red diff.	compression (JPEG, video), skin detection	<code>cv2.COLOR_BGR2YCrCb</code>
CMYK	cyan, magenta, yellow, key	printing only	—

Quiz: colour spaces

QUESTION 1

Why is OpenCV's hue range **0–179** and not 0–359?

An 8-bit channel holds at most 255, so the 360° hue angle is **halved**.

QUESTION 2

You must find a **yellow** line that is partly in **shadow**. Which channels would you try?

H (+ S) in HSV/HLS, or **b*** in LAB — they describe colour separately from brightness.

QUESTION 3

Why does JPEG convert images to **YCbCr** before compressing?

The eye sees fine detail in **brightness (Y)** more than in colour, so Cb and Cr can be stored at lower resolution.

Filters

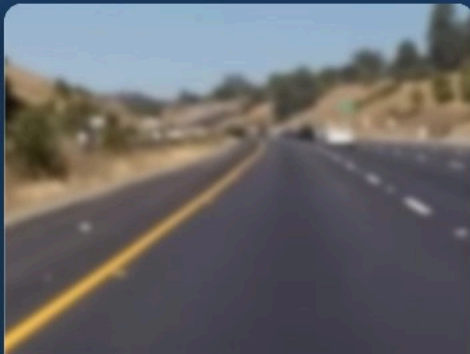
Slide a small **kernel** over every pixel and replace it with a weighted sum of its neighbours.
Same image, different kernel, very different result.



Original
the input image

0	0	0
0	1	0
0	0	0

identity



Gaussian blur
smooth noise & detail

1	2	1
2	4	2
1	2	1

$\div 16$



Sharpen
boost edges & detail

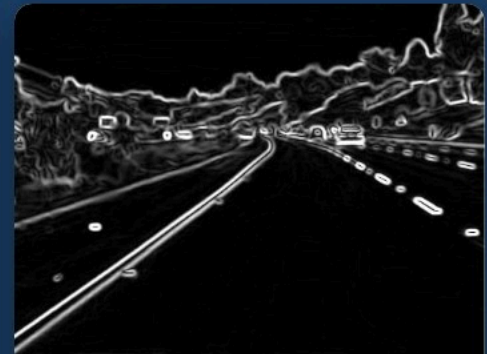
0	-1	0
-1	5	-1
0	-1	0

centre –
neighbours



Median
remove salt-and-pepper noise

median
of 5x5 non-linear

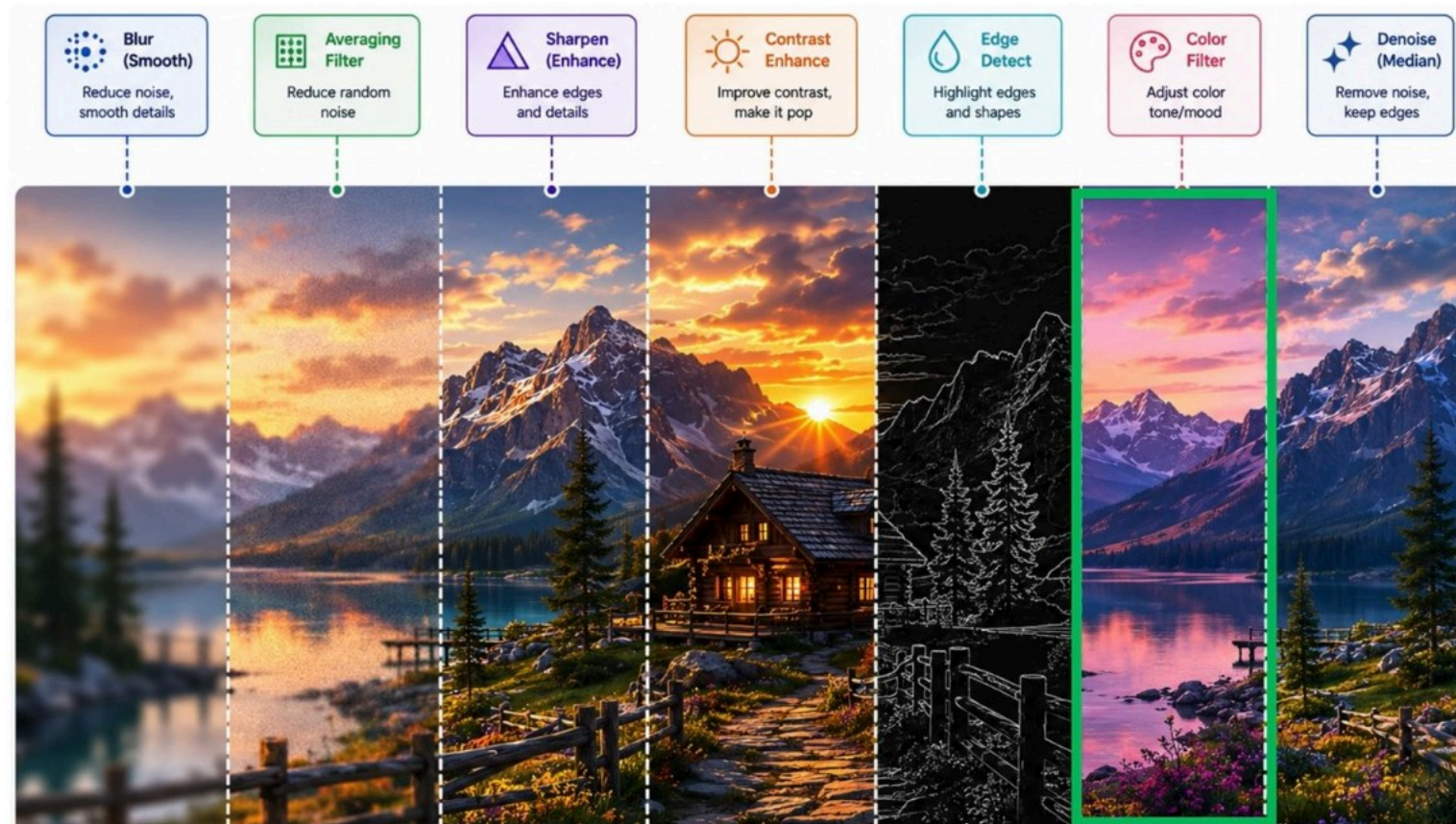


Sobel edges
find where brightness jumps

-1	0	1
-2	0	2
-1	0	1

x-gradient

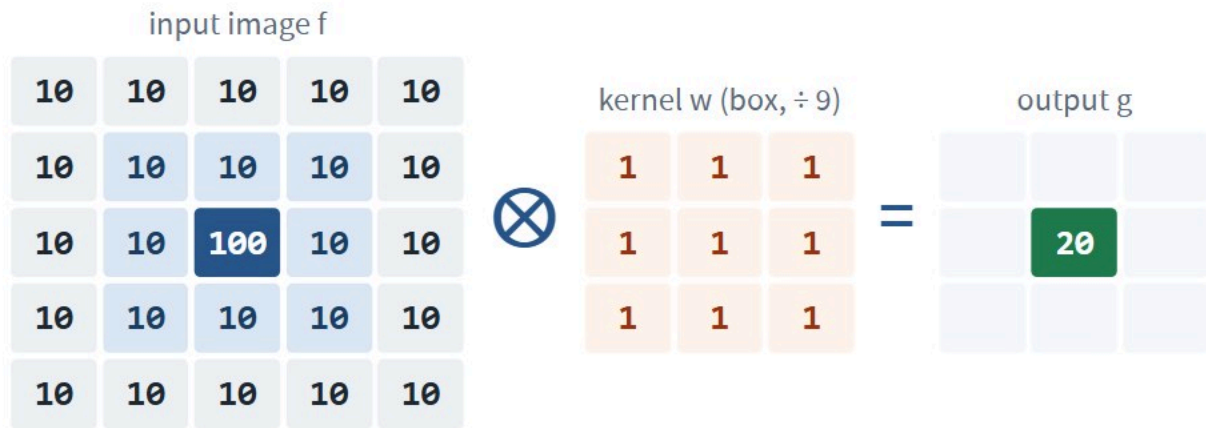
Types of image filters



Neighbourhood filters (this lecture) look at a pixel and its neighbours: blur, sharpen, edges, median.

Point operations look at **one pixel at a time**: contrast, brightness, colour tone (Lecture 3).

How a filter works: convolution



$(8 \times 10 + 100) \div 9 = 180 \div 9 = 20$. The bright spike is spread out and dimmed: that is blurring.

1. Place the kernel over a pixel.
2. Multiply each neighbour by the kernel weight under it.
3. Add them up → the new value of the centre pixel.
4. Slide to the next pixel and repeat — **for every pixel**.

$$g(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k w(i, j) f(x + i, y + j)$$

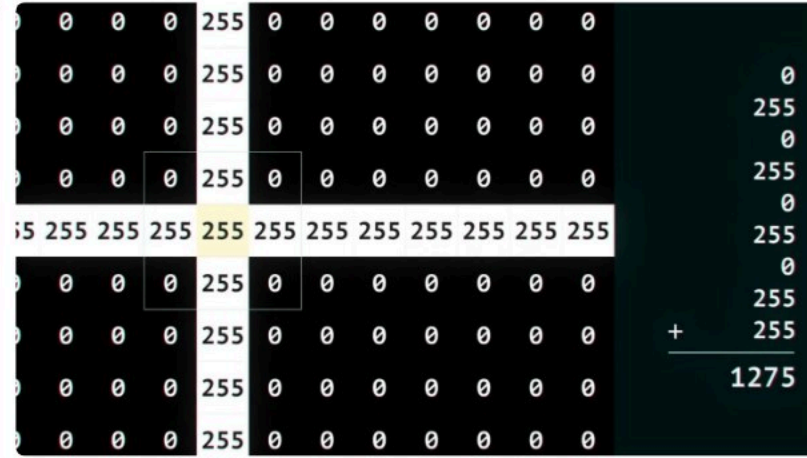
Strictly, this is correlation; convolution flips the kernel first. For symmetric kernels they are identical. At the borders OpenCV reflects the image (`BORDER_REFLECT_101`).

```
k = np.ones((3, 3), np.float32) / 9
out = cv2.filter2D(img, -1, k) # any kernel
```

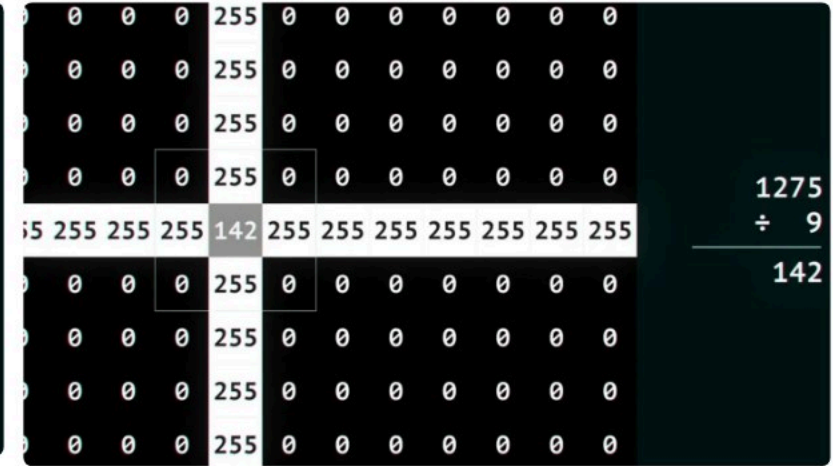
The averaging filter, worked by hand



① a white cross (255) on black (0); 3 × 3 window on the centre



② add the 9 values: $5 \times 255 = 1275$



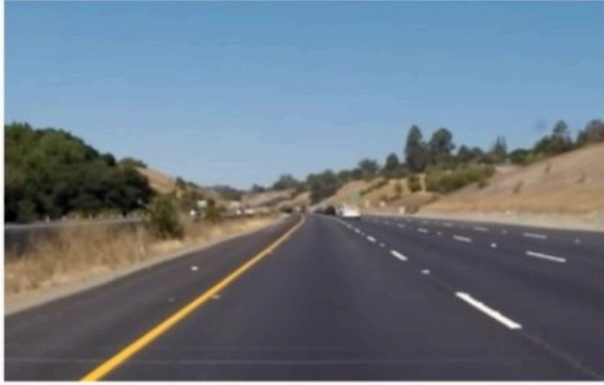
③ divide by 9: $1275 \div 9 = 141.7 \rightarrow 142$

The centre drops from 255 to **142**, and the black pixels next to the line become gray. Repeat everywhere: the sharp cross becomes a soft, wider blur. Averaging removes **noise** — but also **detail**.

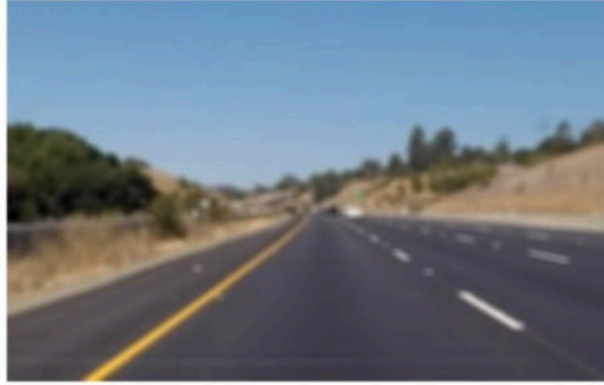
Kernel size matters



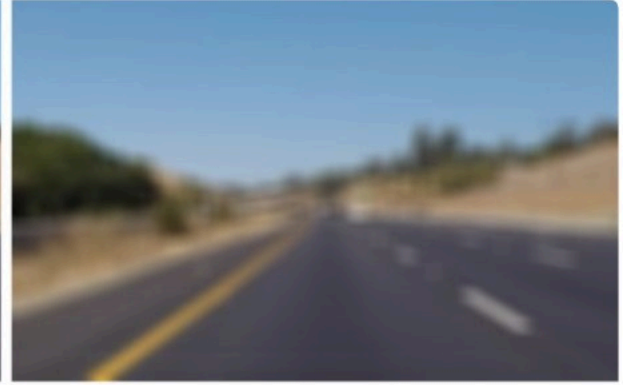
original



blur 3x3



blur 9x9



blur 21x21

```
for k in (3, 9, 21):  
    out = cv2.blur(img, (k, k))    # k x k box filter
```

- Bigger kernel → **more smoothing**, more detail lost.
- Sizes are **odd** (3, 5, 7...) so there is a centre pixel.
- Cost grows with k^2 — use the smallest size that works.

Gaussian blur: closer neighbours count more



Instead of equal weights, use a bell curve:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

1	2	1
2	4	2
1	2	1

3 × 3 Gaussian kernel, ÷ 16.
Centre weight 4, corners 1.

- σ (sigma) controls the strength of the blur.
- Smoother and more natural than a box filter, with no “blocky” artefacts.
- Standard first step before edge detection.

```
out = cv2.GaussianBlur(img, (5, 5), 0)  # sigma 0 → from
```

Median blur: take the middle value

21	28	33	37	39	37	37	15
28	28	33	38	39	32	24	16
32	32	255	255	255	39	24	21
33	38	255	255	255	25	24	27
33	35	255	255	255	32	29	28
27	27	35	34	32	29	29	32
26	26	27	32	30	30	30	33
							33
							38

1. Collect the 9 values in the 3×3 window.
 2. Sort them.
 3. Take the **middle** (5th) value: here 28.
- **Non-linear**: no kernel weights, no sum.
 - One extreme value (a 255 speck) can never become the answer → **perfect for salt-and-pepper noise**.
 - **Keeps edges sharp**, because the output is always a value that really occurs in the window.

```
out = cv2.medianBlur(img, 5)  # kernel size must be odd
```

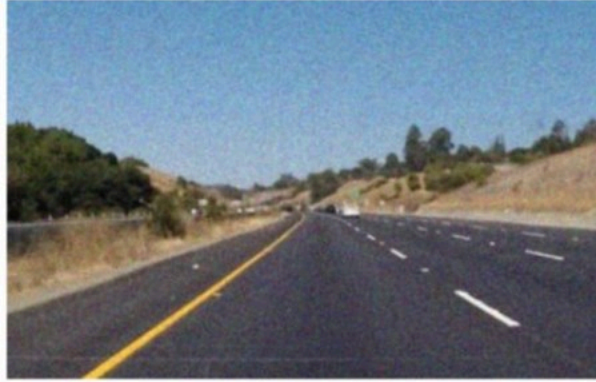
Which blur for which noise?



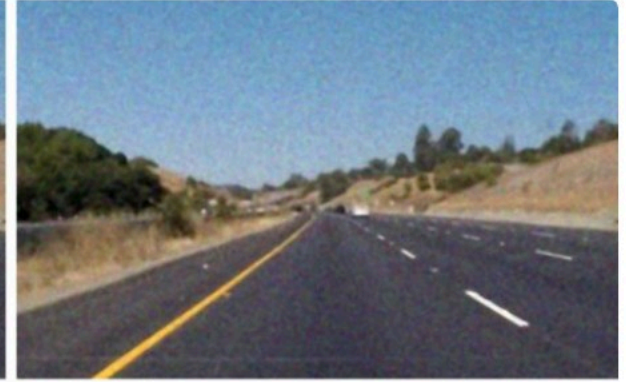
INPUT: Gaussian noise



averaging 5x5



Gaussian 5x5 (best)



median 5



INPUT: salt & pepper noise



averaging 5x5: smeared dots



Gaussian 5x5: still dotted



median 5 (best)

Gaussian noise (every pixel a bit off) → **Gaussian blur** averages it away.

Salt & pepper (a few pixels completely wrong) → **median** ignores the outliers.

Kernel playground: design your own filter



0	0	0
0	1	0
0	0	0

÷ ☐ |abs| for edges

identity

box blur

Gaussian

sharpen

Sobel x

Sobel y

Laplacian

emboss

Left: input · right: output. Edit any number and the output updates. Try: what happens if the weights do not add up to the divisor?

Quiz: filters by hand

A 3×3 neighbourhood:

10	12	11
13	250	12
10	11	13

The centre 250 is a noise speck.

QUIZ QUESTION

1. What does a 3×3 **averaging** filter output for the centre?
2. What does a 3×3 **median** filter output?
3. Which one removed the speck?

- ① $\text{sum} = 342 \rightarrow 342 \div 9 = 38$ (the speck is smeared into its neighbours)
- ② sorted: 10 10 11 11 12 12 13 13 250 $\rightarrow 12$
- ③ the **median** — the outlier never reaches the middle.

05

Edges, shapes & contours

From brightness changes to object outlines

An edge is a big change in brightness

Remember the lane-line patch: 81, 79, 113, 168, 200. Edges are where the image changes fast — the **derivative** (gradient) is large.

Sobel x — finds vertical edges

-1	0	1
-2	0	2
-1	0	1

Sobel y — finds horizontal edges

-1	-2	-1
0	0	0
1	2	1

$$|\nabla f| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan \frac{G_y}{G_x}$$

Three classic edge detectors

- **Sobel** — first derivative in x and y; gives strength and direction.
- **Laplacian** — second derivative; finds edges in all directions at once, but very sensitive to noise.
- **Canny** — blur → Sobel → thin the edges → keep strong edges plus weak ones connected to them. Clean, one-pixel-wide edges. (Full details in [Part II.](#))

Edge detectors compared



original



Sobel x (vertical edges)



Sobel y (horizontal edges)



gradient magnitude



Laplacian (2nd derivative)



Canny (thin, clean)

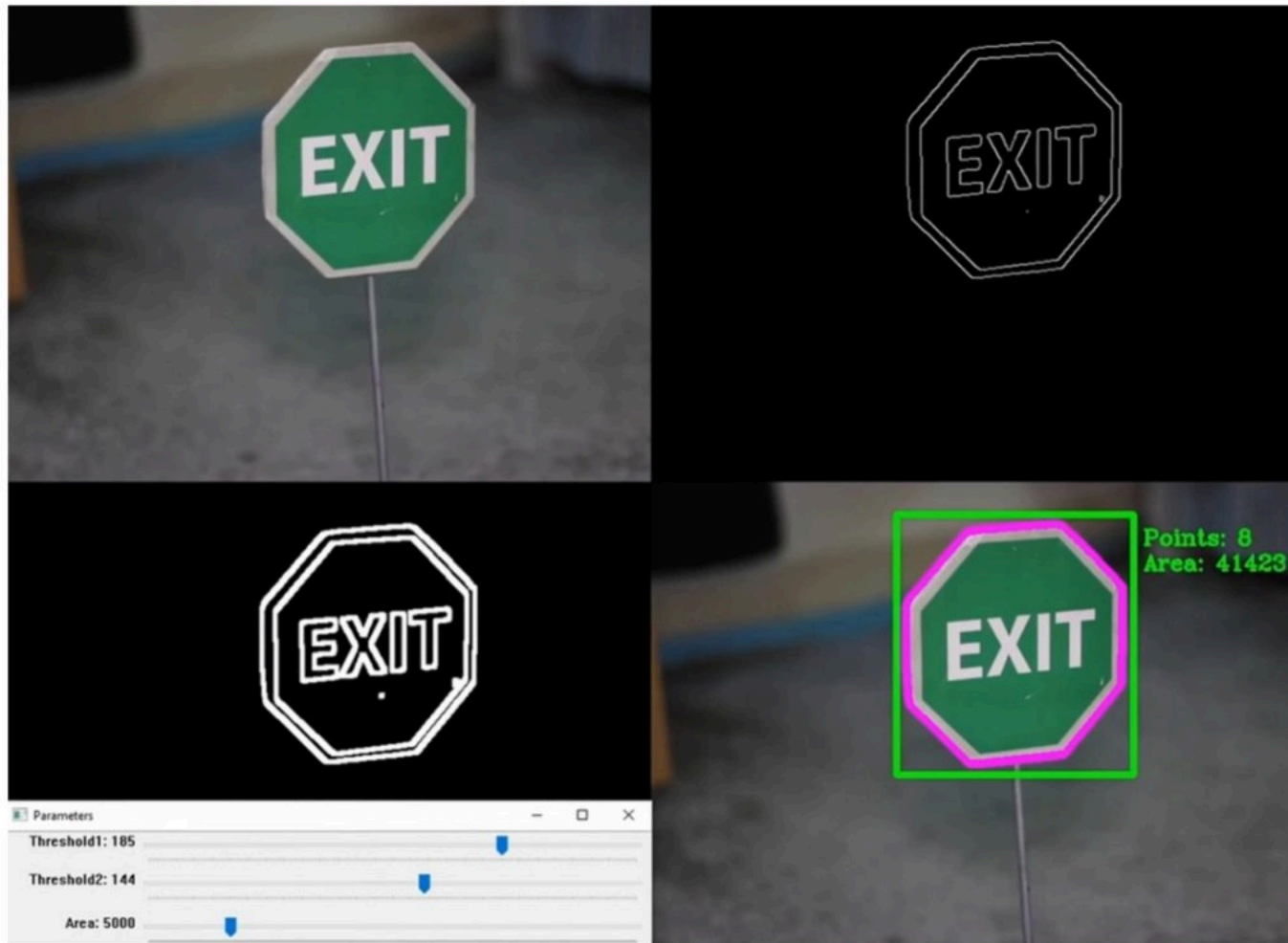
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (3, 3), 0)
```

```
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = cv2.magnitude(sx, sy)
```

```
lap = cv2.Laplacian(gray, cv2.CV_64F)
can = cv2.Canny(gray, 80, 180)
```

Use `CV_64F`, not `uint8`: a dark→bright edge is positive and bright→dark is negative; `uint8` would clip it to 0 and lose half of the edges.

From edges to shapes



The classic shape pipeline:

gray + Gaussian blur

Canny edges

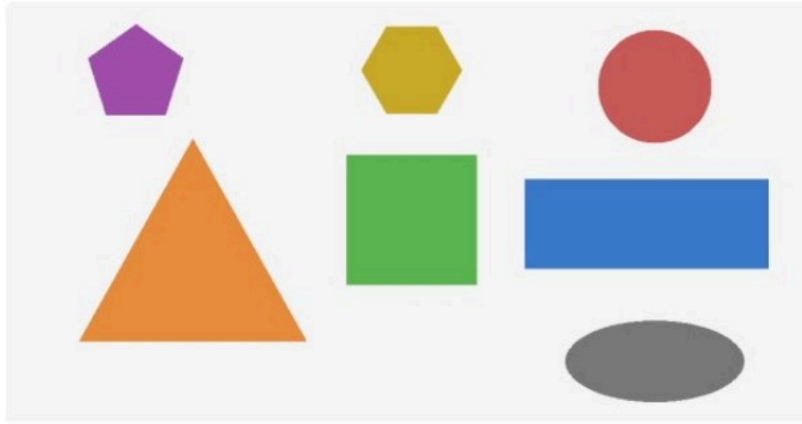
dilate — close small gaps

findContours — outlines

approxPolyDP — count corners

The EXIT sign has **8 corners** → octagon. The trackbars set the two Canny thresholds and the minimum area.

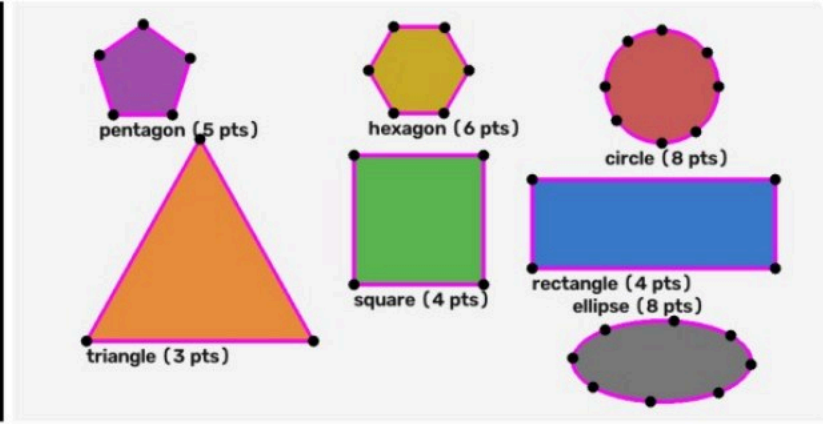
Contours: classify a shape by counting its corners



1. input



2. threshold (binary mask)



3. findContours + approxPolyDP

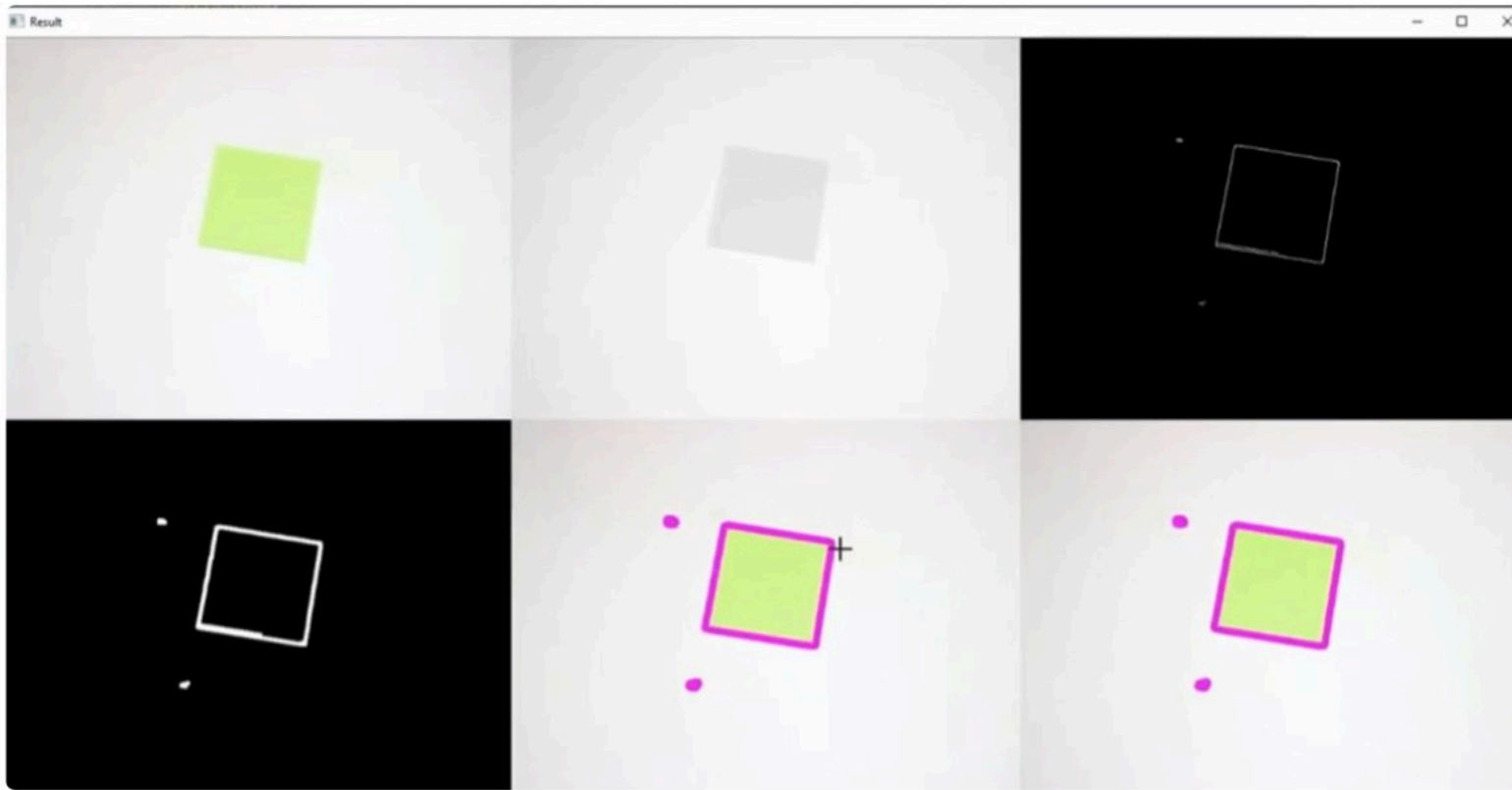
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY_INV)
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)

for c in cnts:
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True)
    corners = len(approx)          # 3 triangle, 4 square/rect, ...
    x, y, w, h = cv2.boundingRect(approx)
```

A **contour** is the list of boundary points of a white region in a binary mask.

- **approxPolyDP** simplifies the outline to a polygon; $\epsilon = 2\%$ of the perimeter.
- 4 corners: square if $w \approx h$, else rectangle.
- Many corners $\rightarrow$ circle or ellipse.

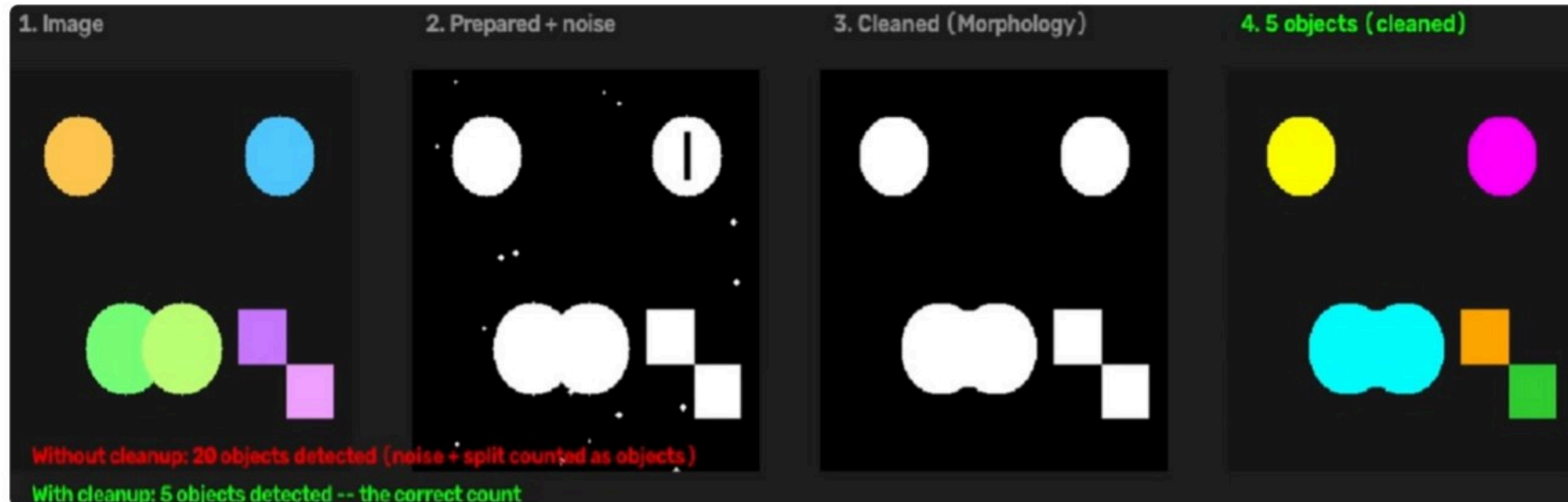
Contours step by step



Top row → bottom row:

1. original image
2. gray + blur
3. Canny edges
4. dilated edges (gaps closed)
5. contour with its **corner points** — small specks are also found
6. final result: only contours **larger than a minimum area** are kept

Clean the mask before you count



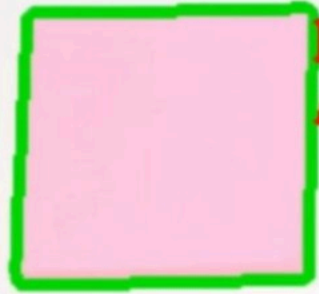
```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k) # remove specks
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k) # fill holes
n = len(cv2.findContours(mask, cv2.RETR_EXTERNAL,
                        cv2.CHAIN_APPROX_SIMPLE)[0])
```

Without cleanup: 20 “objects” — every speck of noise and every split piece count as separate objects.

With morphology: 5 objects, the correct count.

Opening removes specks; **closing** fills small holes.

Contour features: points and area



Points: 4
Area: 12853



Points: 5
Area: 1936



Points: 6
Area: 1760

```
for c in cnts:
    area = cv2.contourArea(c)
    if area < 1000: # ignore noise
        continue
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True)
    cv2.drawContours(img, [c], -1, (0, 255, 0), 3)
    txt = f"Points: {len(approx)} Area: {int(area)}
    cv2.putText(img, txt, (x, y), font, 1, red, 2)
```

Area and corner count are simple **features**: numbers that describe an object. They work for clean shapes — but can they tell two different **logos** apart? →

Quiz: edges & shapes

QUESTION 1

Why do we usually **blur** before detecting edges?

Derivatives amplify **noise**; a Gaussian blur removes it first, so we only get real edges.

QUESTION 2

How many points should `approxPolyDP` return for a **STOP** sign?

8 — a STOP sign is an octagon (like the EXIT sign).

QUESTION 3

Your object counter reports 37 objects for 5 coins. What is the most likely fix?

Clean the mask with **morphological opening** and ignore contours below a **minimum area**.

06

Feature extraction

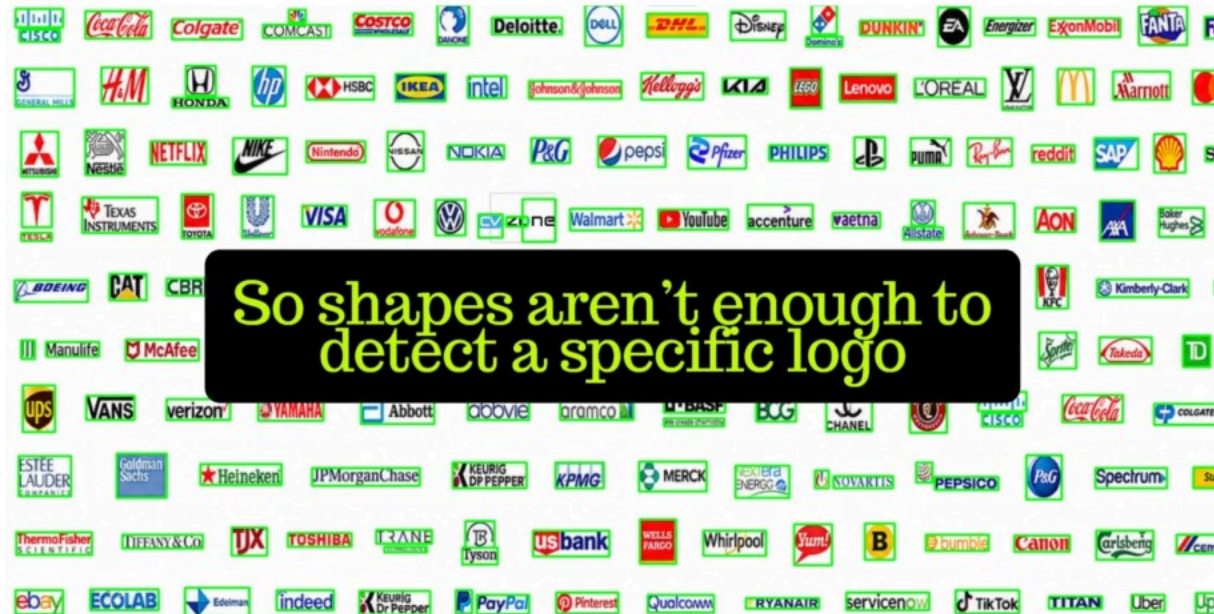
When shapes are not enough

Shapes aren't enough to find a specific logo



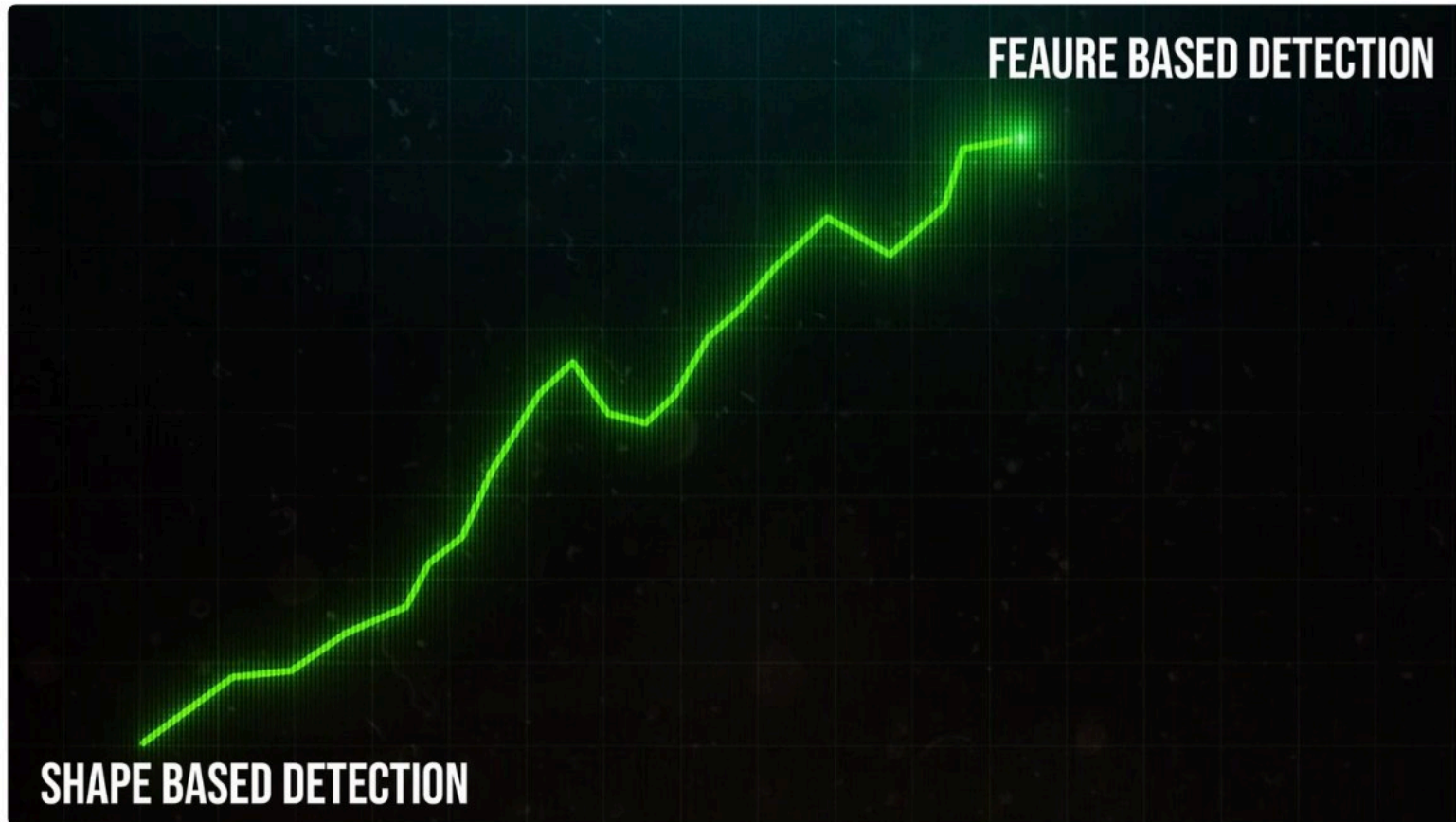
Hundreds of logos — most are circles, rectangles or words.

To recognise one particular object we need **distinctive local patterns** — corners, textures, letter shapes — that appear on that object and nowhere else. These are **features**.



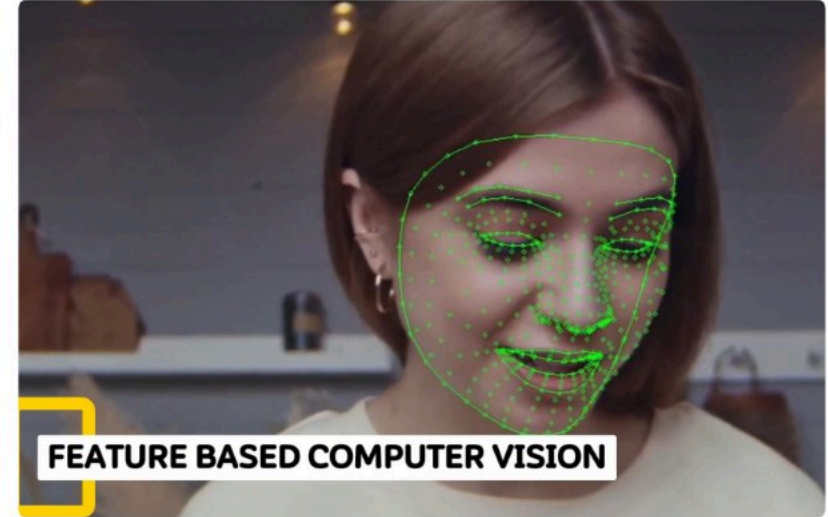
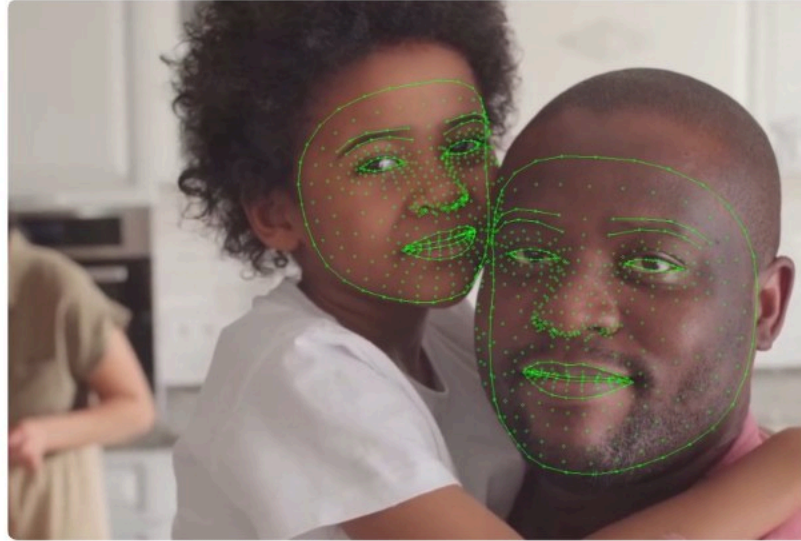
A shape detector boxes every logo — it cannot say which one is which.

Shape-based vs feature-based detection



	Shape-based	Feature-based
Looks at	outline, corners, area	many small, distinctive patches
Finds	“a square”, “an octagon”	“ this logo”, “ this face”
Rotation / scale	fragile	robust (with the right descriptor)
Occlusion	fails if the outline breaks	works with some features hidden

Features in action: face landmarks

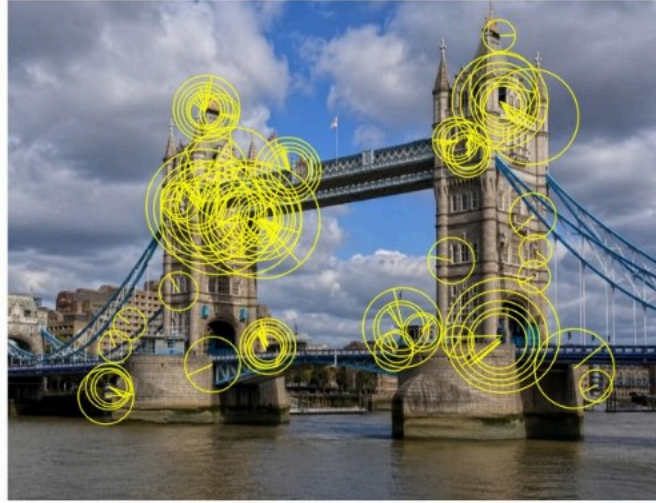


Hundreds of **landmarks** — eye corners, lip contours, nose, jaw line — are found and tracked in real time (here at 32 fps). They work across poses, expressions, ages and skin tones, and drive face filters, driver-drowsiness alerts and avatar animation.

What is a feature? A keypoint plus a descriptor



Shi-Tomasi corners: WHERE



ORB keypoints: where + size + orientation

① Keypoint — where

A point that can be found again: **corners** are good, edges are ambiguous, flat areas are useless. ORB also stores a **size** and an **orientation**.

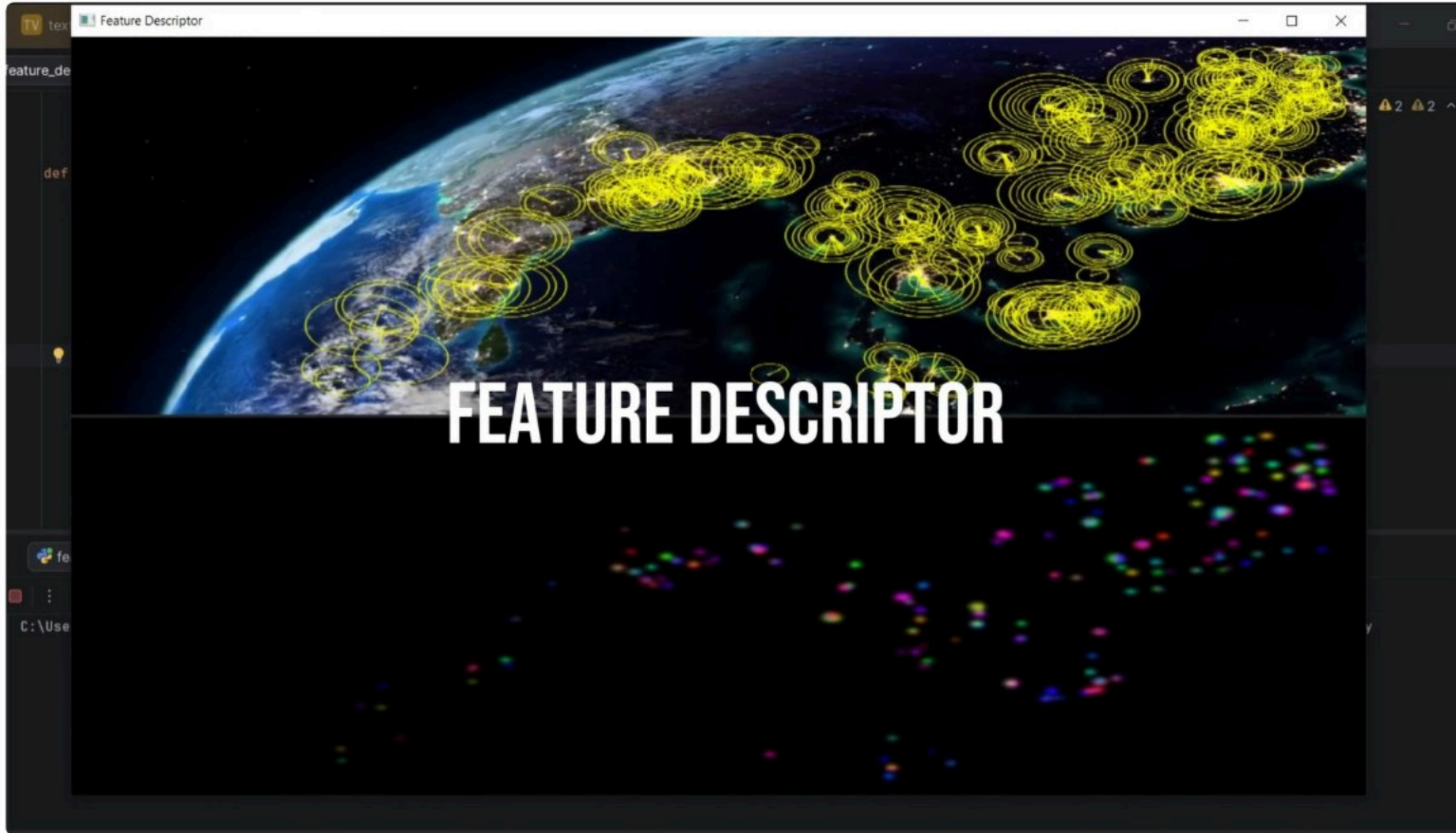
② Descriptor — what

A vector that summarises the patch around the keypoint. ORB uses **256 bits**; SIFT uses 128 numbers.

Good features are ...

repeatable, **distinctive**, and **invariant** to rotation, scale and lighting.

Features describe the whole scene



Each yellow circle is a keypoint; its size is the scale at which it was found. Below, every keypoint becomes a **descriptor** — a point in a high-dimensional space.

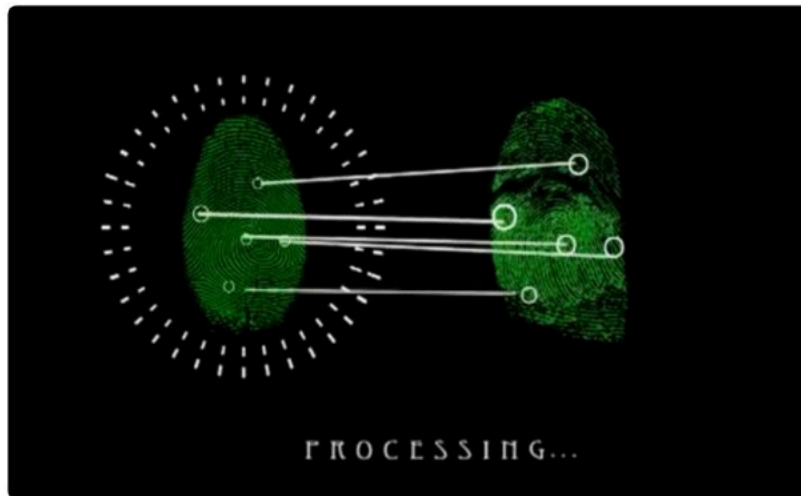
Classic detectors and descriptors:

- **Harris** (1988) — corners
- **SIFT** (2004) — scale and rotation invariant; free in OpenCV since its patent expired in 2020
- **ORB** (2011) — fast, binary, free: the default choice on a Raspberry Pi

Matching features works like fingerprint matching



① find distinctive points (minutiae)



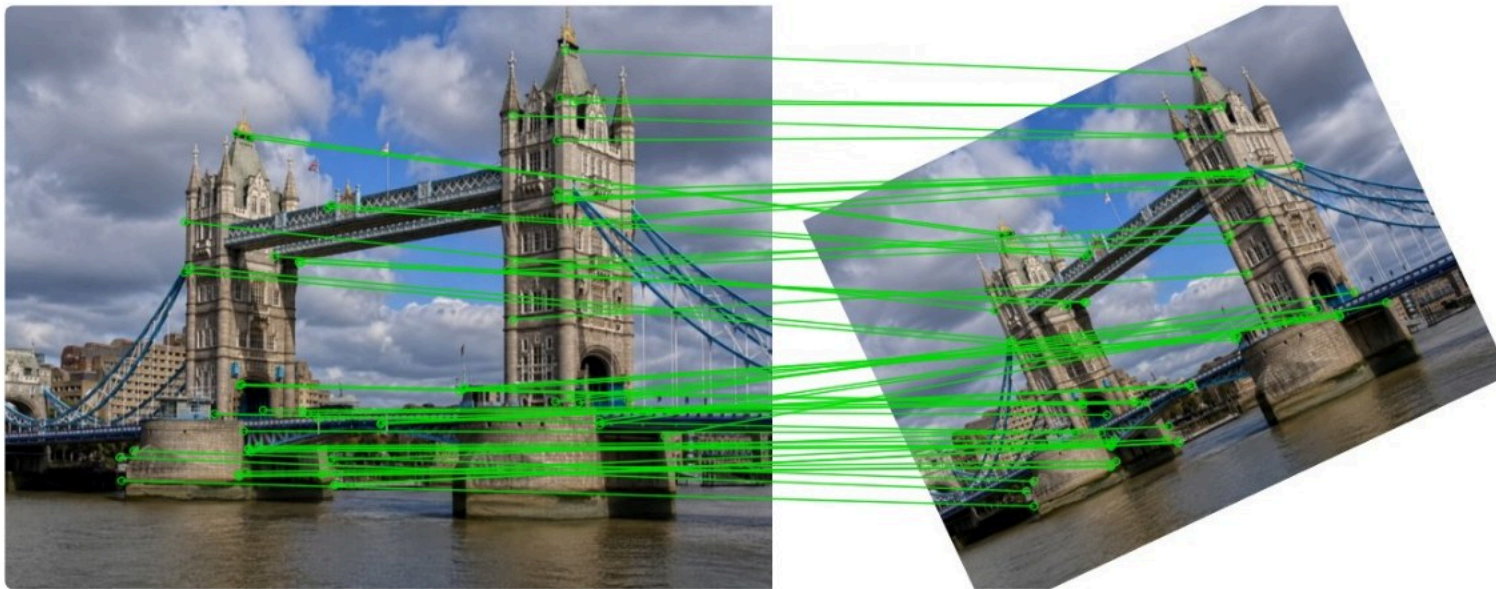
② compare their descriptions with the database



③ enough consistent matches → **same finger**

Image matching does the same: compare descriptors, keep the nearest neighbours, and check that the matches agree on one geometric transformation.

ORB feature matching in OpenCV



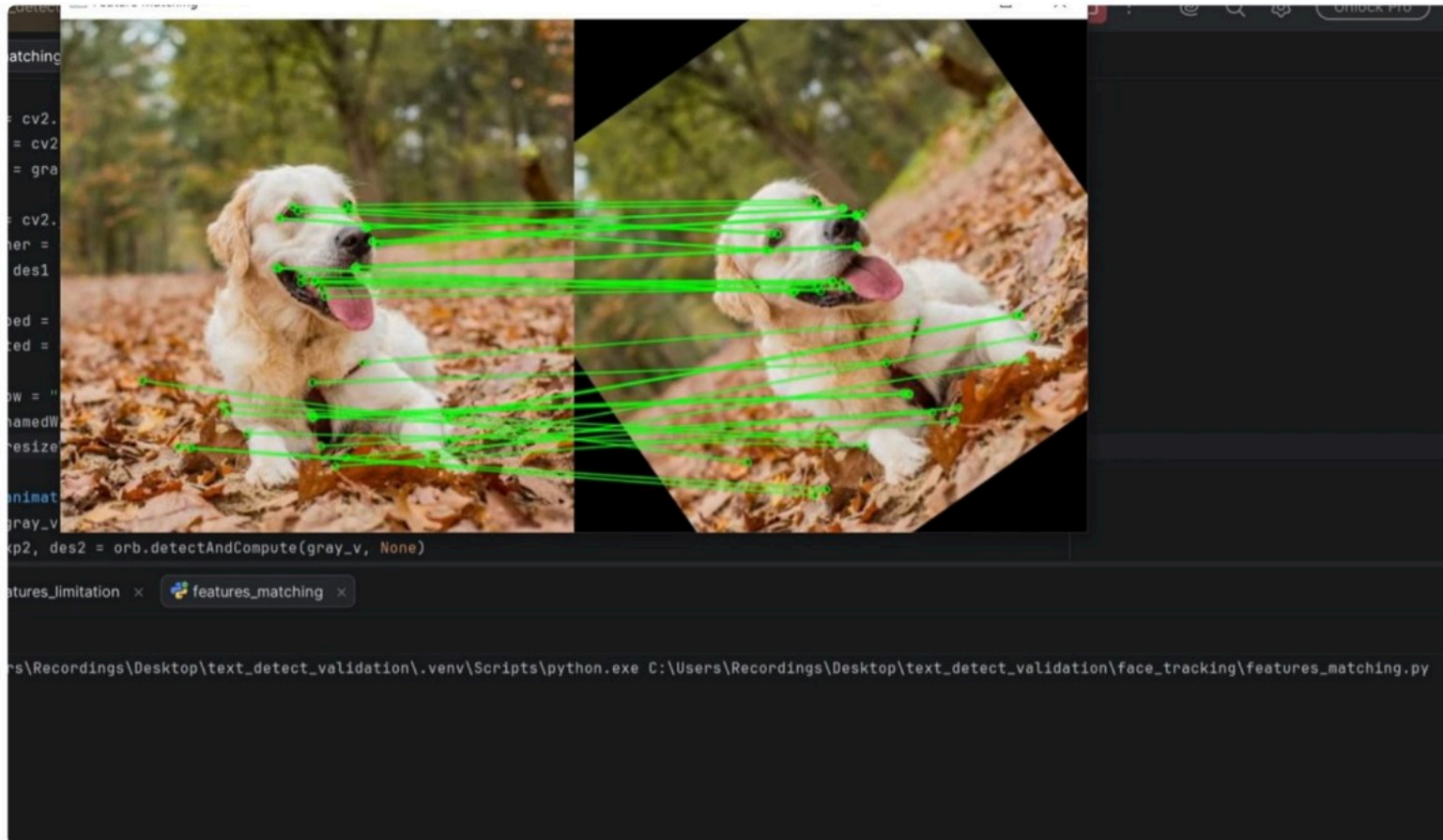
```
orb = cv2.ORB_create(1500)
k1, d1 = orb.detectAndCompute(gray1, None)
k2, d2 = orb.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING)
good = [m for m, n in bf.knnMatch(d1, d2, k=2)
        if m.distance < 0.75 * n.distance] # ratio test
H, inl = cv2.findHomography(pts1, pts2, cv2.RANSAC, 5.0)
```

The right image is **rotated 25°** and **scaled to 75%**.
Still:

- 1500 keypoints per image
- **541** pass the ratio test
- **534** agree on one transformation (RANSAC)

Classroom demo: matching survives rotation



The same ORB pipeline on a photo and a rotated copy of it. Most green lines connect the **same** point on the dog — the ear, the eye, the nose.

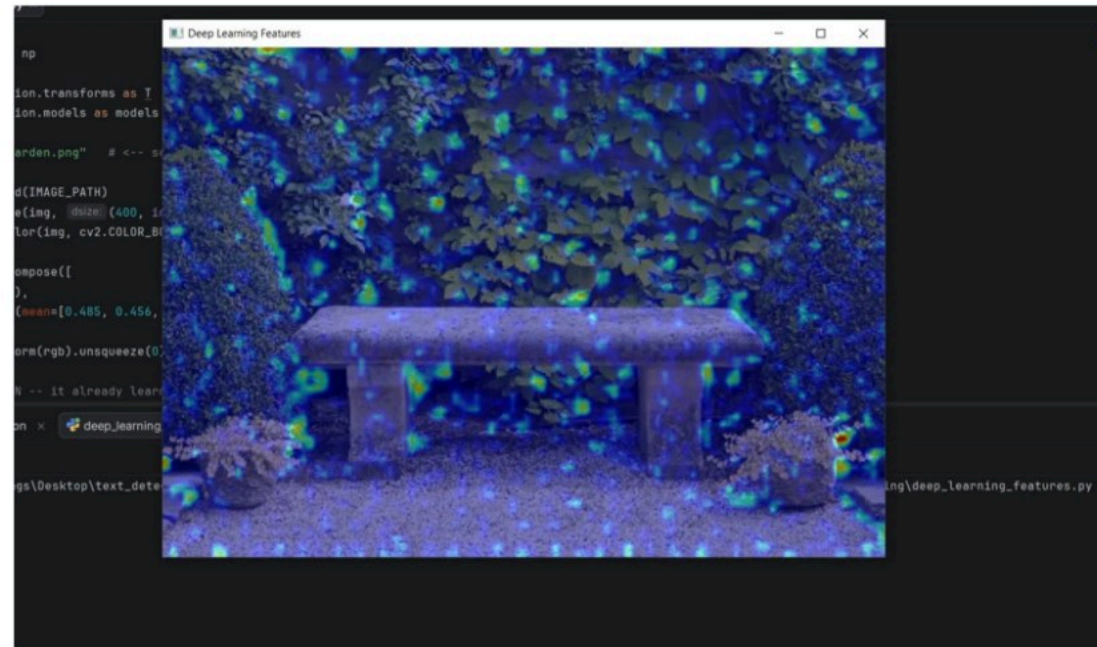
A few lines cross the image: these are **wrong matches** (outliers). That is why we use the ratio test and RANSAC.

Applications: panorama stitching, visual odometry and SLAM, augmented reality, object instance recognition.

Hand-crafted vs learned features



Hand-crafted: a corner detector designed by people finds bench corners and leaf tips.



Learned: a pretrained CNN's feature map — the network decided for itself which patterns matter.

Hand-crafted features (Harris, SIFT, ORB) follow rules we wrote. Deep networks **learn** their features from millions of examples — which brings us to the last section.

Quiz: features

QUESTION 1

Why is a **corner** a better keypoint than a point on a straight **edge**?

Along an edge every point looks the same (it can slide). A corner changes in **every** direction, so it can be located exactly.

QUESTION 2

What is the difference between a **keypoint** and a **descriptor**?

Keypoint = **where** (x, y, scale, angle). Descriptor = **what** the patch looks like (a vector we can compare).

QUESTION 3

Why do we use **NORM_HAMMING** to match ORB descriptors?

ORB descriptors are **binary** strings; Hamming distance (count of differing bits) is the right, very fast measure.

07

Deep-learning-based computer vision

Let the machine learn the features

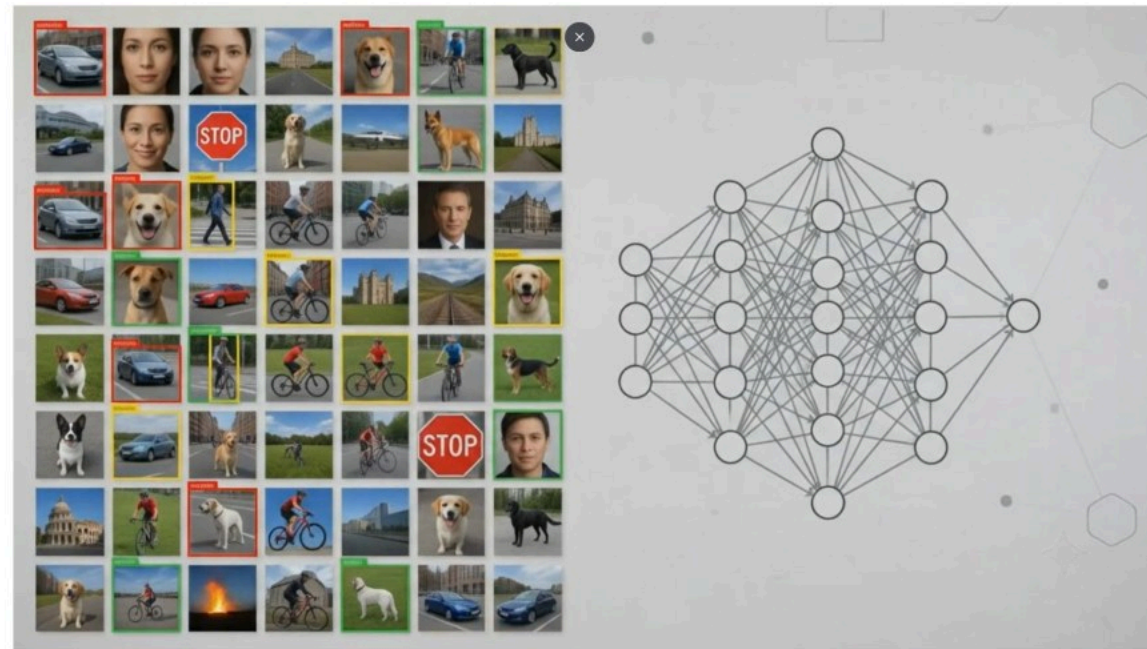
The diagram illustrates the Machine Learning process. On the right, a stylized brain icon is labeled "Machine". Below the brain is a green progress bar indicating 100% completion. The text "Machine is learning itself" is written below the progress bar. On the left, a code editor window titled "New Training - Test Training" displays the following Python code:

```
1 import cv2
2
3 image = cv2.imread('objact.jpg', cv2.IMREAD_GRAYSCALE)
4
5 # For Resize to pick convy
6 image = cv2.resize(image, (100, 100))
7
8 # For Convolve -> pick kernel
9 kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
10
11 # For Laplace operation -> pick filter
12 filter = cv2.LAPLACE
13 features = cv2.filter2D(image, -1, filter)
```

A person chooses the colour space, the thresholds, the kernels and the features. Everything in sections 2–6 of this lecture.

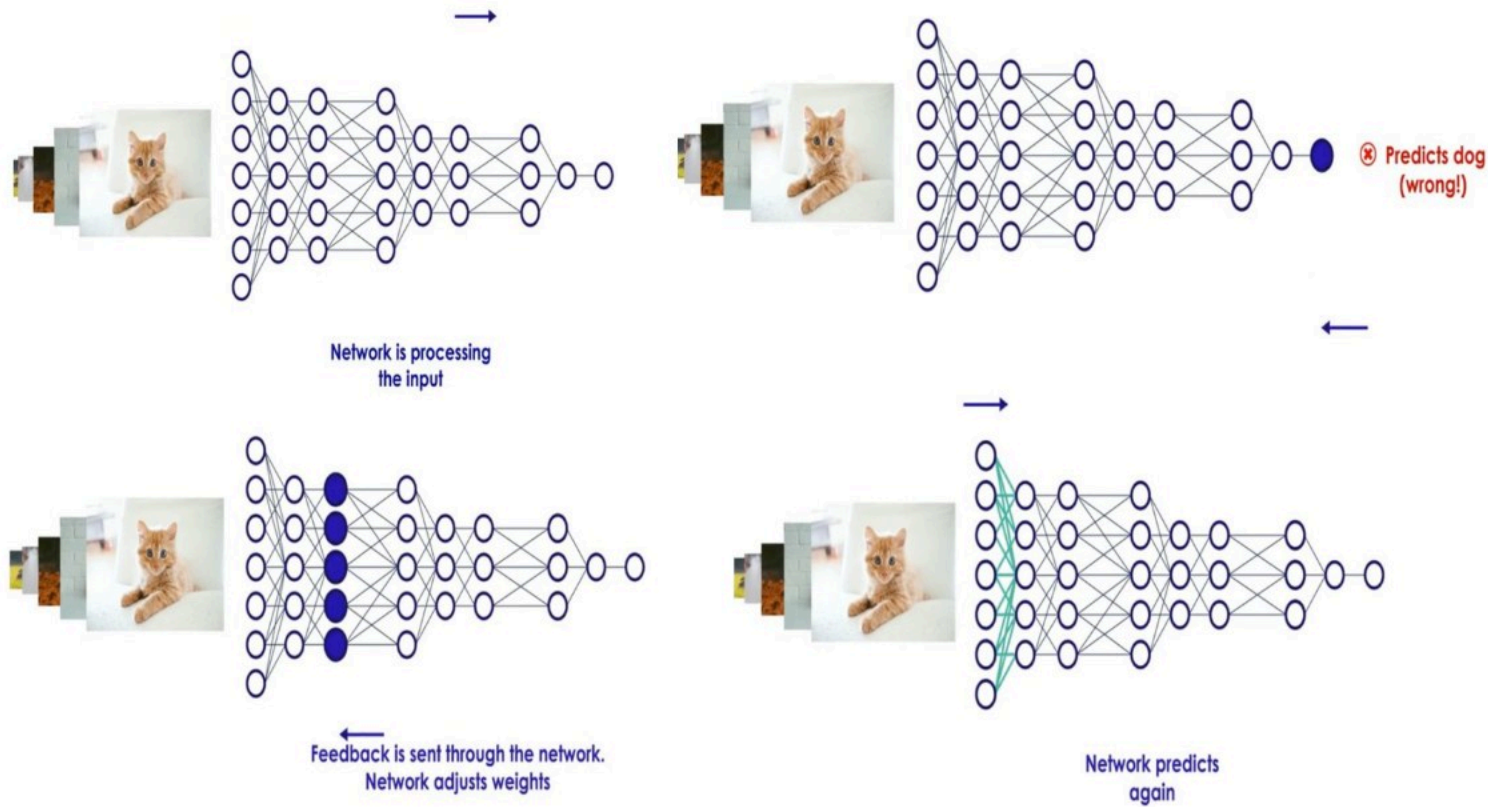
We provide **labelled examples**; the network learns its own kernels and features by trial and error.

Performing a task without explicit programming

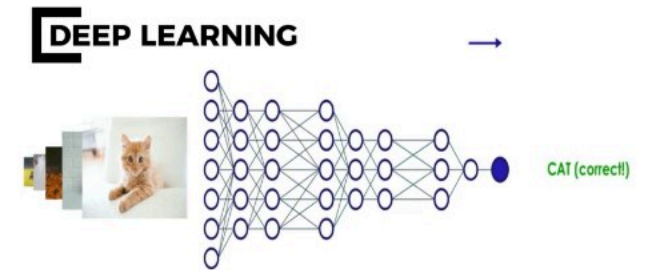


Machine learning is the “field of study that gives computers the ability to learn without being explicitly programmed” (definition often attributed to Arthur Samuel, 1959). For vision, the program is replaced by a **dataset**: thousands of images of cars, dogs, bicycles, STOP signs and faces, each with its label.

How a network learns: predict, compare, adjust

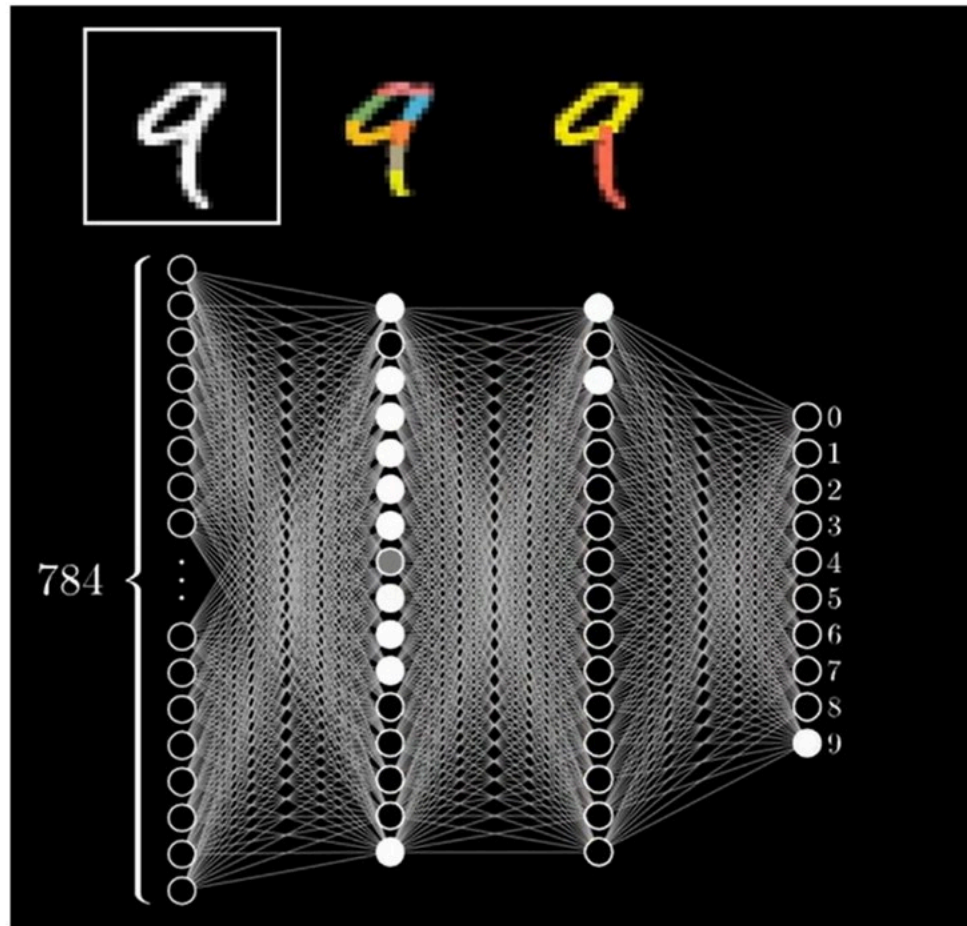


1. **Forward pass:** the cat image flows through the layers.
2. The network predicts “**dog**” — **wrong**.
3. The **loss** measures the error; **back-propagation** sends it backwards and every weight is nudged a little.
4. Predict again. Repeat over millions of images.



After training: “CAT (correct!)”

Recognising handwritten digits: still just numbers

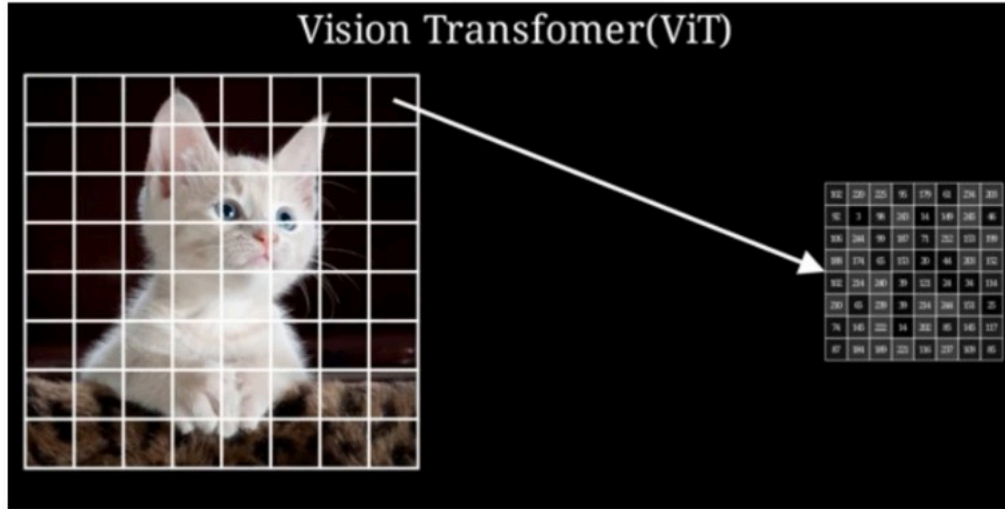


- Input: a 28×28 grayscale image = **784 numbers** (section 2!).
- Hidden layers combine pixels into **strokes**, then strokes into **loops and lines**.
- Output: **10 neurons**, one per digit 0–9; the brightest wins — here “9”.

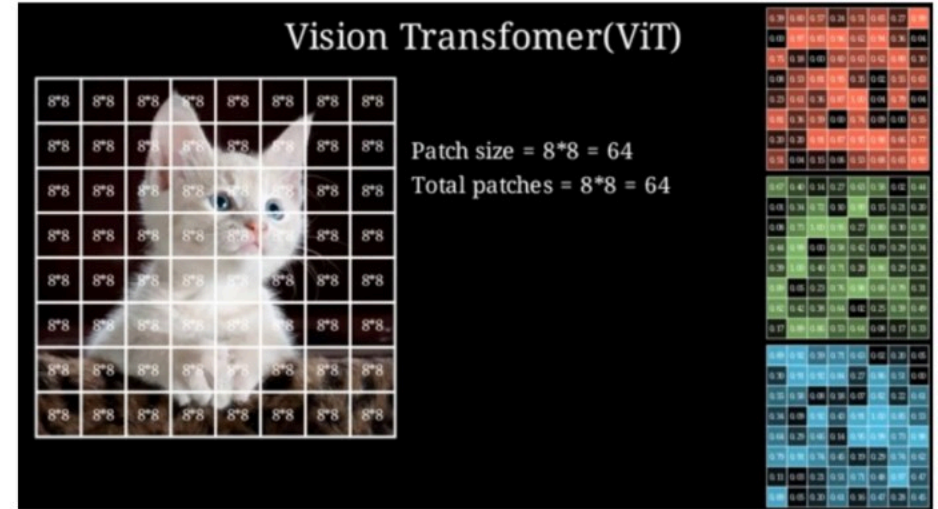
The link to today

A **convolutional** network (CNN) learns 3×3 **kernels** just like the ones in section 4. Its first layer typically ends up with Sobel-like edge detectors and blob detectors — learned, not designed.

Vision Transformer (ViT): an image as patches



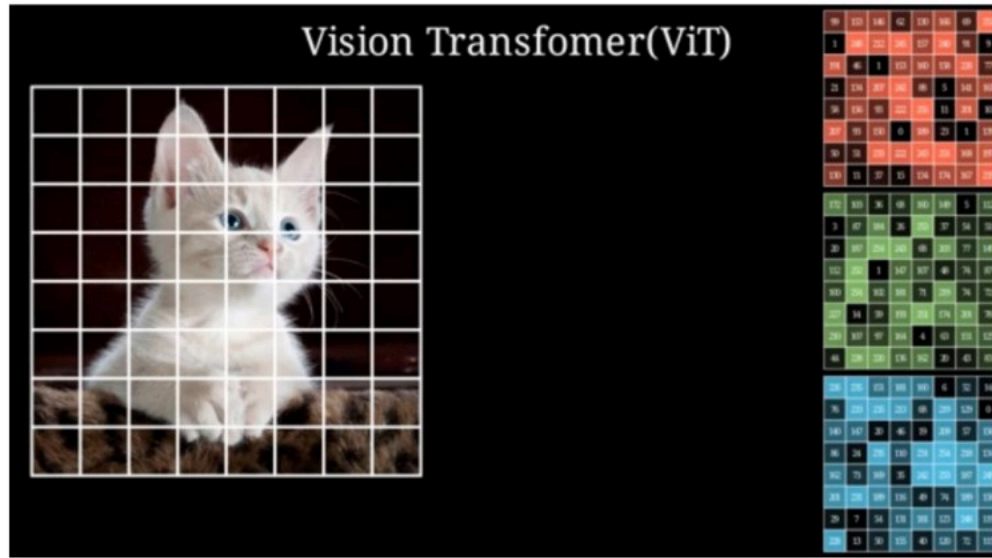
split the image into patches; each patch is a block of numbers



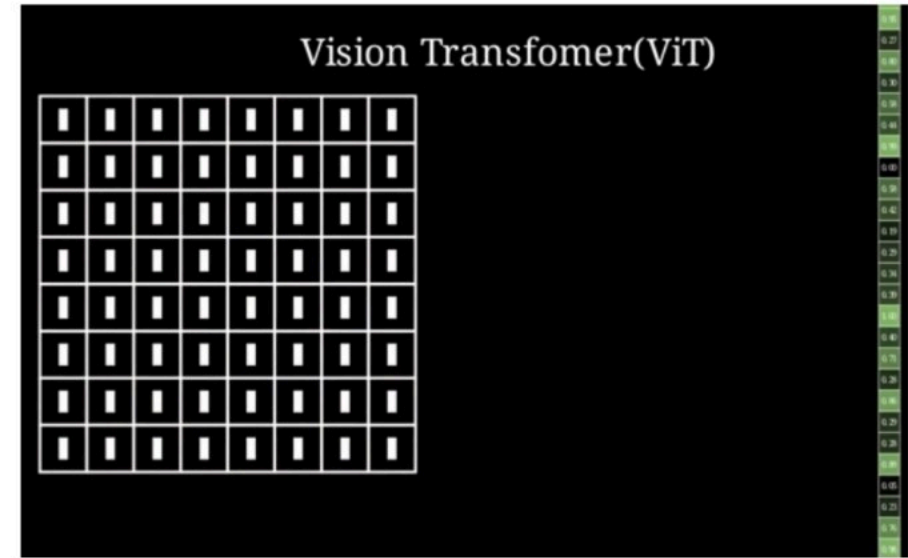
8×8 grid $\rightarrow$ 64 patches

Transformers were built for **words**. ViT (Dosovitskiy et al., 2021, “An Image is Worth 16×16 Words”) treats each image **patch** as a **word**: flatten it, map it to a vector (a **token**), add its position, and let self-attention relate every patch to every other patch.

ViT: patches become numbers, numbers become tokens



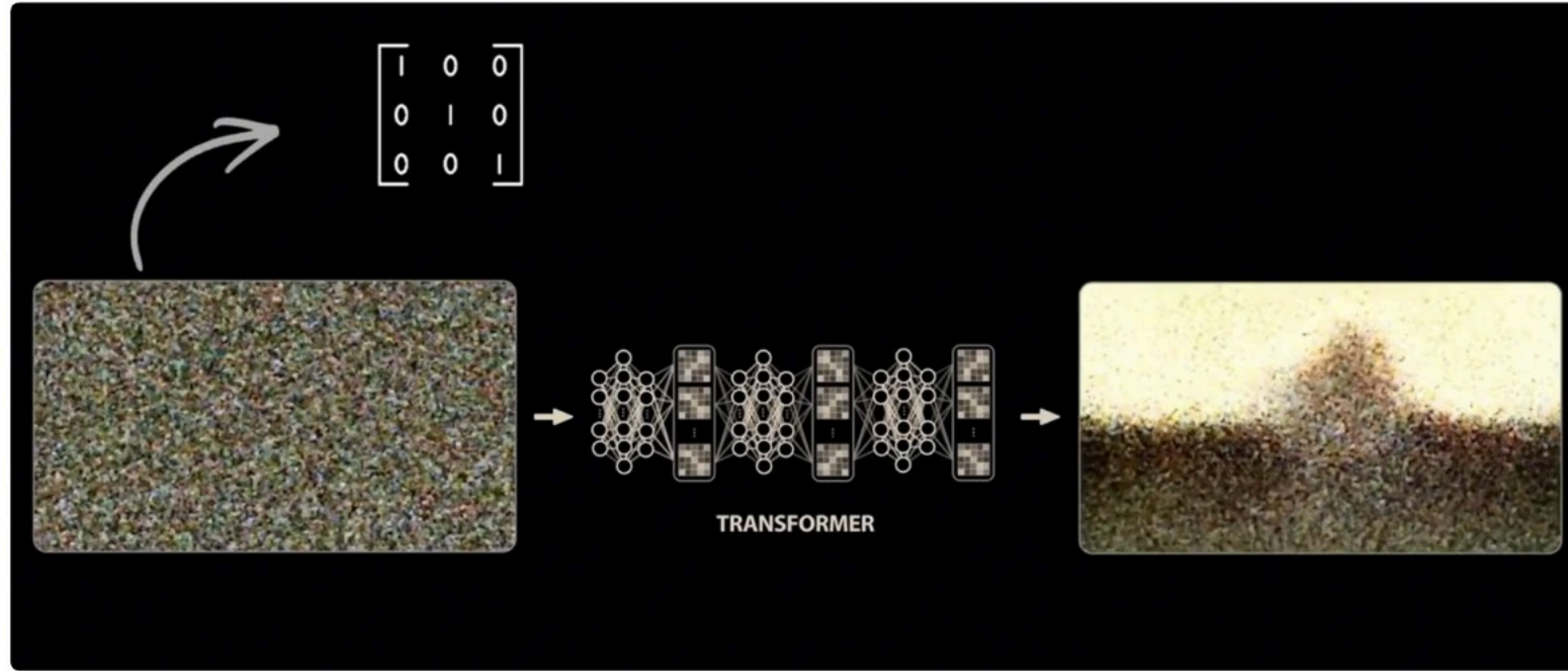
every patch = R, G, B numbers (section 2 again)



each patch $\rightarrow$ one embedding vector (token)

A 224×224 RGB image with 16×16 patches $\rightarrow 14 \times 14 = 196$ tokens, each made from $16 \times 16 \times 3 = 768$ numbers.

Running filters backwards: from noise to an image



In section 4 we **removed** noise. **Generative models** (diffusion models with transformer backbones) learn to do this so well that they can start from **pure noise** and remove it step by step, until an image appears that matches a text prompt.

Classical vs deep-learning computer vision

	Classical (sections 2–6)	Deep learning (section 7)
Features	designed by people: colour thresholds, kernels, ORB	learned from data
Data needed	a few images to tune parameters	thousands to millions of labelled images
Compute	tiny — runs on a microcontroller	a GPU to train; often an accelerator to run
Explainable?	yes — every step can be inspected	harder — feature maps, attention maps
Robustness	breaks when conditions change (light, weather)	generalises well if the training data is diverse
Today	pre-processing, measurement, embedded systems	classification, detection, segmentation, generation

Real systems use **both**: classical pre-processing (colour conversion, resizing, normalisation) feeds every deep network.

Key takeaways

② Images are numbers

$H \times W \times C$ arrays of `uint8` 0–255.
OpenCV uses **BGR**.

③ Colour spaces

Same information, different axes. HSV/LAB separate colour from brightness.

④ Filters

A kernel slides over every pixel.
Gaussian for Gaussian noise, median for salt & pepper.

⑤ Edges & shapes

Edges = large gradients.
Contours + `approxPolyDP` count corners.

⑥ Features

Keypoint (where) + descriptor (what). Match with ratio test + RANSAC.

⑦ Deep learning

Networks learn the kernels and features themselves. ViT turns patches into tokens.

OpenCV cheat sheet for Part I

```
img  = cv2.imread("img.jpg")          # BGR
rgb  = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv  = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, lower, upper)
out  = cv2.bitwise_and(img, img, mask=mask)

box  = cv2.blur(img, (5, 5))
gaus = cv2.GaussianBlur(img, (5, 5), 0)
med  = cv2.medianBlur(img, 5)
any_ = cv2.filter2D(img, -1, kernel)
```

```
sx  = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
lap = cv2.Laplacian(gray, cv2.CV_64F)
edge = cv2.Canny(gray, 50, 150)

k    = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k)
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)
peri   = cv2.arcLength(c, True)
approx = cv2.approxPolyDP(c, 0.02 * peri, True)

orb = cv2.ORB_create()
kp, des = orb.detectAndCompute(gray, None)
```

Practice & resources

Practise in the labs (Colab-ready)

- [Lab 4A · An image really is just numbers](#)
- [Lab 5A · Colour selection](#)
- [Lab 5C · Canny edge detection](#)
- [Lecture 5 Part II · Finding lane lines](#)

Your turn: take a photo of objects on your desk, count them with threshold → morphology → contours, then match one object between two photos with ORB.

Further reading

- Szeliski, Computer Vision: Algorithms and Applications 2e — ch. 3 (image processing), ch. 7 (feature detection & matching)
- OpenCV tutorials: [colour spaces](#) · [smoothing](#) · [contours](#) · [ORB](#)
- Dosovitskiy et al., [An Image is Worth 16×16 Words](#), ICLR 2021

Questions?

Next: **Part II — Computer Vision for Self-Driving Cars**